

医院大当家

CrazyHospital (工程代号) • V2.0.0

Source code excerpt (first 30 + last 30 pages)

Scope: Assets_Game own source. Files: 41 / Lines: 9218 / 50 lines per page / Total pages: 185

Generated: 2026-10-10 23:25:42 (local build chain, headless Edge print)

```
// ===== FILE: Assets\_Game\Scripts\Editor\AlbumCheck.cs =====
// AlbumCheck.cs - the three-volume album + wonder-drug research gate
// (G09-T009 acceptance, batchmode entry):
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyHospital.AlbumCheck.Run -logFile <abs>
// SectionA: the frozen volume-table mirror (three volumes in pack order,
//           the 5/6/3 entry counts, the membership resolver, the research
//           gold closed set + the +2% per-volume income law).
// SectionB: the rate + bonus solvers (floor-percent collection rate, the
//           zero-total guard, the one-below boundary never completes,
//           the pure bonus solver equals the live income face).
// SectionC: the unlock face (evt_album_unlock funnel line, the
//           double-unlock and unknown-entry refusals, the patient-intake
//           alias, the evt_album_complete once-guard crossing).
// SectionD: the research line (the 100/250/500 gold closed set, the
//           evt_research funnel, the short-gold / unknown-row /
//           already-researched refusals, the three-domain bonus books).
// SectionE: the doctor-volume fold (SyncDoctors rides silent on the hire
//           face - zero duplicate unlock lines, the six-card volume
//           completes once, idempotence, the null-desk guard).
// SectionF: the inheritance + identity + zero-RNG parity (the second-run
//           re-seed keeps the earned volumes silently, a fresh album
//           books nothing, the double run is byte-identical).
// Exit codes: 0=all green 1=fail/exception
using System.Collections.Generic;
using System.IO;
using System.Text;
using Sy.Platform;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyHospital
{
    public static class AlbumCheck
    {
        static readonly List<string> Lines = new List<string>();
        static int _failCount;

        const string DayKey = "20991231";

        public static void Run()
        {
            int code;
            try
            {
                SectionA_FrozenVolumeMirror();
                SectionB_RateBonusSolvers();
                SectionC_UnlockFace();
                SectionD_ResearchLine();
                SectionE_DoctorVolumeFold();
                SectionF_InheritanceIdentityParity();
            }
        }
    }
}
```

```

        code = _failCount == 0 ? 0 : 1;
    }
    catch (System.Exception e)
    {
        Lines.Add("[FAIL] exception: " + e);
        code = 1;
    }

    WriteReport();
    foreach (var line in Lines) Debug.Log(line);
    EditorApplication.Exit(code);
}

// ---- SectionA: the frozen volume-table mirror ----
static void SectionA_FrozenVolumeMirror()
{
    var vols = HospitalAlbum.VolumeIds();
    bool order = vols.Length == 3
        && vols[0] == "album_disease" && vols[0] == HospitalAlbum.DiseaseVolumeId
        && vols[1] == "album_doctor" && vols[1] == HospitalDoctors.DoctorAlbumId
        && vols[2] == "album_drug" && vols[2] == HospitalAlbum.DrugVolumeId;
    bool counts = HospitalData.AlbumVolumeCount == 3
        && HospitalAlbum.TotalCountOf("album_disease") == 5
        && HospitalAlbum.TotalCountOf("album_doctor") == 6
        && HospitalAlbum.TotalCountOf("album_drug") == 3;

    // the catalog face: EntriesOf returns the frozen table rows in pack order
    var dis = HospitalAlbum.EntriesOf("album_disease");
    bool disRows = dis.Count == 5
        && dis[0] == "dis_hiccup_bug" && dis[1] == "dis_gummy_bone" && dis[2] == "dis_monday"
        && dis[3] == "dis_head_grass" && dis[4] == "dis_cactus_accident";
    var doc = HospitalAlbum.EntriesOf("album_doctor");
    bool docRows = doc.Count == 6
        && doc[0] == "doc_threeseec" && doc[1] == "doc_remedy_quack" && doc[2] == "doc_moyu"
        && doc[3] == "doc_wangdali" && doc[4] == "doc_night_nurse" && doc[5] == "doc_beauty_chief";
    var drug = HospitalAlbum.EntriesOf("album_drug");
    bool drugRows = drug.Count == 3
        && drug[0] == "drug_hiccupCola" && drug[1] == "drug_sleepwalk_patch" && drug[2] == "drug_shock_sugar";
    PassFail("A1 frozen volume mirror: three volumes in pack order (diseases/doctors/wonder drugs), the 5/6/3 entry counts
and the catalog rows read the frozen tables verbatim (numeric pack s6/s10 + M07/M08)",
        order && counts && disRows && docRows && drugRows);

    // the membership resolver: every frozen entry resolves to its volume, anything else resolves to ""
    bool member = HospitalAlbum.VolumeOf("dis_hiccup_bug") == "album_disease"
        && HospitalAlbum.VolumeOf("doc_moyu") == "album_doctor"
        && HospitalAlbum.VolumeOf("drug_shock_sugar") == "album_drug"
        && HospitalAlbum.VolumeOf("dis_nope") == "" && HospitalAlbum.VolumeOf("") == ""
        && HospitalAlbum.VolumeOf(null) == "";

    // the research closed set: the gold ladder 100/250/500, one bonus domain each, +1 magnitude
    var rs = HospitalData.Research();

```

```

    bool gold = rs.Length == 3
        && rs[0].id == "drug_hiccup_cola" && rs[0].cost == 100 && rs[0].bonusId == "reg_fee_pct" && rs[0].bonusValue == 1
        && rs[1].id == "drug_sleepwalk_patch" && rs[1].cost == 250 && rs[1].bonusId == "bed_fee_pct" && rs[1].bonusValue == 1
        && rs[2].id == "drug_shock_sugar" && rs[2].cost == 500 && rs[2].bonusId == "cure_rate" && rs[2].bonusValue == 1
        && rs[0].albumId == "album_drug" && rs[1].albumId == "album_drug" && rs[2].albumId == "album_drug";
    bool incomeLaw = HospitalData.AlbumBonusPerVolumePct == 2; // the section-five closed set: +2% income per completed
volume
    PassFail("A2 closed sets: the research gold ladder 100/250/500 with one bonus domain each (reg_fee/bed_fee/cure_rate +1)
and the completed-volume income law +2% (frozen, zero new numbers)",
        member && gold && incomeLaw);
}

// ---- SectionB: the rate + bonus solvers ----
static void SectionB_RateBonusSolvers()
{
    bool solver = HospitalAlbum.SolveRatePct(0, 0) == 0
        && HospitalAlbum.SolveRatePct(0, 5) == 0
        && HospitalAlbum.SolveRatePct(1, 3) == 33 // floor percent
        && HospitalAlbum.SolveRatePct(2, 3) == 66
        && HospitalAlbum.SolveRatePct(3, 3) == 100
        && HospitalAlbum.SolveRatePct(4, 5) == 80
        && HospitalAlbum.SolveRatePct(5, -1) == 0; // a broken total guards to zero
    bool liveFace = true;
    foreach (var v in HospitalAlbum.VolumeIds())
    {
        var alb = new HospitalAlbum();
        var ids = HospitalAlbum.EntriesOf(v);
        alb.ArmForGate(ids.ToArray());
        if (alb.RatePctOf(v) != 100 || !alb.IsComplete(v)) liveFace = false;
    }
    PassFail("B1 rate solver: floor percent at every probe and the broken-total guard reads zero; the armed full set reads
100% complete through the live face",
        solver && liveFace);

    // the one-below boundary: a single missing entry keeps the volume partial - only the rate face moves, the income face
stays flat
    bool oneBelow = true;
    foreach (var v in HospitalAlbum.VolumeIds())
    {
        var alb = new HospitalAlbum();
        var ids = HospitalAlbum.EntriesOf(v);
        var sub = new List<string>();
        for (int i = 0; i < ids.Count - 1; i++) sub.Add(ids[i]);
        alb.ArmForGate(sub.ToArray());
        if (alb.IsComplete(v) || alb.RatePctOf(v) != HospitalAlbum.SolveRatePct(sub.Count, ids.Count) || alb.IncomeBonusPct
!= 0) oneBelow = false;
    }
    var albD = new HospitalAlbum();
    var docIds = HospitalAlbum.EntriesOf("album_doctor");
    var subDocs = new List<string>();
    for (int i = 0; i < 5; i++) subDocs.Add(docIds[i]);
    albD.ArmForGate(subDocs.ToArray());
    bool partialLaw = albD.RatePctOf("album_doctor") == 83 && !albD.IsComplete("album_doctor") && albD.IncomeBonusPct == 0;
// the deepest one-below probe: 5/6 doctors
    PassFail("B2 one-below boundary: every volume at total-1 stays partial (rate moves, income +2% never pays early)",
        oneBelow && partialLaw);

```

```

// the pure bonus solver equals the live income face at every completed count 0..3
bool bonus = true;
for (int n = 0; n <= 3; n++)
    if (HospitalAlbum.SolveBonusPct(n) != n * HospitalData.AlbumBonusPerVolumePct) bonus = false;
var desk = NewDesk();
desk.InheritFrom(HospitalAlbum.EntriesOf("album_doctor"));
var alb3 = new HospitalAlbum(desk);
alb3.ArmForGate(HospitalAlbum.EntriesOf("album_disease").ToArray());
alb3.ArmForGate(HospitalAlbum.EntriesOf("album_drug").ToArray());
alb3.SyncDoctors();
bool threeVolumes = alb3.CompletedVolumeCount == 3 && alb3.IncomeBonusPct == 3 * HospitalData.AlbumBonusPerVolumePct;
PassFail("B3 bonus accounting: the pure solver pays +2% per completed volume and the three-volume walk folds 6% through
the live face (income reads completed count only)",
    bonus && threeVolumes);
}

// ---- SectionC: the unlock face ----
static void SectionC_UnlockFace()
{
    // the comedy-events face: an unlock books exactly one funnel line
    var alb = new HospitalAlbum();
    bool first = alb.Unlock("dis_hiccup_bug")
        && alb.Funnel.Count == 1 && alb.Funnel[0] == "evt_album_unlock@album_disease:dis_hiccup_bug";
    bool dup = !alb.Unlock("dis_hiccup_bug") && alb.Funnel.Count == 1; // the double unlock refuses, no duplicate line
    bool unknown = !alb.Unlock("dis_nope") && !alb.Unlock("") && alb.Funnel.Count == 1; // unknown ids never enter a volume
    bool rate = alb.RatePctOf("album_disease") == 20; // 1/5 = 20% floor
    PassFail("C1 unlock face: evt_album_unlock@<volume>:<entry> books once, the double-unlock and unknown-entry refusals
change nothing, the rate reads 1/5 = 20%",
        first && dup && unknown && rate);

    // the patient-intake face is the same unlock (M08: the absurd disease registers the moment its patient books in)
    var albI = new HospitalAlbum();
    bool alias = albI.RegisterDisease("dis_gummy_bone")
        && albI.Funnel.Count == 1 && albI.Funnel[0] == "evt_album_unlock@album_disease:dis_gummy_bone";
    PassFail("C2 patient-intake alias: RegisterDisease books the same single unlock line (the host wiring rides this face)",
        alias);

    // the complete crossing: the once-guard books evt_album_complete exactly when the volume fills, never twice
    var albC = new HospitalAlbum();
    var dis = HospitalAlbum.EntriesOf("album_disease");
    for (int i = 0; i < dis.Count - 1; i++) albC.Unlock(dis[i]);
    bool partialQuiet = albC.Funnel.Count == dis.Count - 1 && albC.IncomeBonusPct == 0;
    albC.Unlock(dis[dis.Count - 1]);
    bool crossed = albC.IncomeBonusPct == HospitalData.AlbumBonusPerVolumePct
        && CountFunnelTag(albC.Funnel, "evt_album_complete") == 1
        && albC.Funnel[albC.Funnel.Count - 1] == "evt_album_complete@album_disease";
    int funBefore = albC.Funnel.Count;
    albC.Unlock("dis_nope"); // a refused unlock never re-books the crossing
    bool once = albC.Funnel.Count == funBefore && CountFunnelTag(albC.Funnel, "evt_album_complete") == 1;
    PassFail("C3 complete crossing: partial unlocks stay quiet (+0 income), the fifth disease books
evt_album_complete@album_disease exactly once and the once-guard holds",
        partialQuiet && crossed && once);
}

```

```

}

// ---- SectionD: the research line ----
static void SectionD_ResearchLine()
{
    // the happy path: exact gold spent, evt_research booked, the unlock rides the purchase
    var alb = new HospitalAlbum();
    int c1 = alb.TryResearch("drug_hiccup_cola", 100);
    int c2 = alb.TryResearch("drug_sleepwalk_patch", 300);
    int c3 = alb.TryResearch("drug_shock_sugar", 500);
    bool spend = c1 == 100 && c2 == 250 && c3 == 500
        && alb.ResearchSpendHc == 850; // the caller books the spent gold at the wiring slice
    bool fun = Contains(alb.Funnel, "evt_research@drug_hiccup_cola:100")
        && Contains(alb.Funnel, "evt_research@drug_sleepwalk_patch:250")
        && Contains(alb.Funnel, "evt_research@drug_shock_sugar:500")
        && Contains(alb.Funnel, "evt_album_unlock@album_drug:drug_hiccup_cola")
        && CountFunnelTag(alb.Funnel, "evt_research") == 3;
    bool complete = alb.UnlockedCountOf("album_drug") == 3 && alb.IsComplete("album_drug")
        && Contains(alb.Funnel, "evt_album_complete@album_drug")
        && alb.IncomeBonusPct == HospitalData.AlbumBonusPerVolumePct;
    PassFail("D1 research happy path: the 100/250/500 ladder spends exact gold, books evt_research@<drug>:<cost> per row, the
unlock rides each purchase and the third row completes the drug volume (+2% income)",
        spend && fun && complete);

    // the refusals: short gold, unknown row, already researched - all refuse with zero state change
    var albR = new HospitalAlbum();
    bool shortGold = albR.TryResearch("drug_shock_sugar", 499) == 0
        && albR.ResearchSpendHc == 0 && albR.Funnel.Count == 0;
    bool unknownRow = albR.TryResearch("drug_nope", 9999) == 0 && albR.Funnel.Count == 0;
    albR.TryResearch("drug_hiccup_cola", 100);
    int funBefore = albR.Funnel.Count;
    bool already = albR.TryResearch("drug_hiccup_cola", 9999) == 0
        && albR.Funnel.Count == funBefore
        && CountFunnelTag(albR.Funnel, "evt_research") == 1; // no duplicate purchase or unlock line
    PassFail("D2 research refusals: short gold (499<500), unknown row and an already-researched row all return 0 with zero
state change - no phantom spend, no duplicate lines",
        shortGold && unknownRow && already);

    // the three-domain bonus books: each researched row pays its own domain, unresearched domains read zero
    bool books = alb.ResearchBonusOf("reg_fee_pct") == 1
        && alb.ResearchBonusOf("bed_fee_pct") == 1
        && alb.ResearchBonusOf("cure_rate") == 1
        && alb.ResearchBonusOf("nope_domain") == 0;
    var albZ = new HospitalAlbum();
    bool zeroFace = albZ.ResearchBonusOf("reg_fee_pct") == 0 && albZ.ResearchBonusOf("") == 0;
    PassFail("D3 three-domain accounting: reg_fee_pct/bed_fee_pct/cure_rate each read +1 from their researched row, foreign
and empty domains read zero (the intake fold is the wiring slice's call)",
        books && zeroFace);
}

// ---- SectionE: the doctor-volume fold ----
static void SectionE_DoctorVolumeFold()
{

```

```

// the six-card payroll folds silent: the hire face already booked evt_doctor_hire, so the sync books no unlock lines
var desk = NewDesk();
desk.InheritFrom(HospitalAlbum.EntriesOf("album_doctor"));
var alb = new HospitalAlbum(desk);
bool silent = alb.Funnel.Count == 0 && alb.UnlockedCountOf("album_doctor") == 0;
alb.SyncDoctors();
bool folded = alb.UnlockedCountOf("album_doctor") == 6
    && alb.IsComplete("album_doctor")
    && CountFunnelTag(alb.Funnel, "evt_album_unlock") == 0 // silent ride: zero duplicate unlock lines
    && CountFunnelTag(alb.Funnel, "evt_album_complete") == 1
    && Contains(alb.Funnel, "evt_album_complete@album_doctor")
    && alb.IncomeBonusPct == HospitalData.AlbumBonusPerVolumePct;
int funBefore = alb.Funnel.Count;
alb.SyncDoctors(); // idempotence: the roster never double-folds
bool idempotent = alb.Funnel.Count == funBefore && alb.UnlockedCountOf("album_doctor") == 6;
PassFail("E1 doctor-volume fold: the full payroll syncs silent (zero evt_album_unlock lines - the hire face owns that
story), completes the six-card volume once, pays +2% and stays idempotent",
    silent && folded && idempotent);

// a partial payroll stays partial: three cards read 50%, no crossing, no income
var deskP = NewDesk();
var three = new List<string>();
var docIds = HospitalAlbum.EntriesOf("album_doctor");
for (int i = 0; i < 3; i++) three.Add(docIds[i]);
deskP.InheritFrom(three);
var albP = new HospitalAlbum(deskP);
albP.SyncDoctors();
bool partial = albP.RatePctOf("album_doctor") == 50 && !albP.IsComplete("album_doctor")
    && albP.IncomeBonusPct == 0 && albP.Funnel.Count == 0;
PassFail("E2 partial payroll: three of six cards read 50% with no crossing and no income (the volume pays only
complete)",
    partial);

// the null-desk guard: a headless album without a doctors host folds nothing and never crashes
var albN = new HospitalAlbum(null);
albN.SyncDoctors();
bool nullFace = albN.Funnel.Count == 0 && albN.CompletedVolumeCount == 0;
PassFail("E3 null-desk guard: SyncDoctors without a doctors host is a clean no-op",
    nullFace);
}

// ---- SectionF: the inheritance + identity + zero-RNG parity ----
static void SectionF_InheritanceIdentityParity()
{
    // the second-run re-seed: the earned volumes land silently on a fresh run (InheritAlbumsKept)
    var run1 = new HospitalAlbum();
    run1.Unlock("dis_hiccup_bug");
    run1.TryResearch("drug_hiccup_cola", 100);
    var run2 = new HospitalAlbum();
    run2.InheritFrom(run1.Unlocked);
    bool kept = run2.UnlockedCountOf("album_disease") == 1 && run2.UnlockedCountOf("album_drug") == 1
        && run2.RatePctOf("album_disease") == 20

```

```

        && run2.Funnel.Count == 0 // the armed state books no story lines
        && run2.ResearchSpendHc == 0; // the fresh run never re-spends the inherited research
    int keptCount = run2.Unlocked.Count;
    run2.InheritFrom(new[] { "dis_nope", "" }); // unknown rows never enter through inheritance
    bool junk = run2.Unlocked.Count == keptCount;
    run2.InheritFrom(null); // the null list is a clean no-op
    bool nulllist = run2.Unlocked.Count == keptCount;
    PassFail("F1 second-run inheritance: the earned entries re-seed silently (zero funnel lines, zero re-spend), unknown rows
refuse and the null list no-ops",
        kept && junk && nulllist);

    // the identity law: a fresh album books nothing - the passive income face reads zero and never self-books
    var fresh = new HospitalAlbum();
    bool identity = fresh.Unlocked.Count == 0 && fresh.Funnel.Count == 0
        && fresh.CompletedVolumeCount == 0 && fresh.IncomeBonusPct == 0
        && fresh.ResearchSpendHc == 0
        && fresh.RatePctOf("album_disease") == 0 && fresh.RatePctOf("album_doctor") == 0 && fresh.RatePctOf("album_drug") ==
0;
    PassFail("F2 identity: the untouched album reads all-zero through every face (the +2% passive never fires without a
completed volume - the T005 economy stays byte-safe)",
        identity);

    // the double-run byte parity: the same full script fingerprints identically (zero RNG)
    string sigA = FullScript();
    string sigB = FullScript();
    PassFail("F3 double-run byte parity: two identical full scripts (inherit + research + sync) agree byte-identical through
the album fingerprint (sig_len="
        + sigA.Length + ")", sigA == sigB && sigA.Length > 0);
}

/// <summary>The full collection script: the second-run inheritance
/// of the disease volume, the whole research ladder, the full
/// doctor payroll sync - one deterministic fingerprint.</summary>
static string FullScript()
{
    var desk = NewDesk();
    desk.InheritFrom(HospitalAlbum.EntriesOf("album_doctor"));
    var alb = new HospitalAlbum(desk);
    alb.InheritFrom(HospitalAlbum.EntriesOf("album_disease"));
    alb.TryResearch("drug_hiccup_cola", 100);
    alb.TryResearch("drug_sleepwalk_patch", 250);
    alb.TryResearch("drug_shock_sugar", 500);
    alb.SyncDoctors();
    return alb.Fingerprint();
}

static HospitalDoctors NewDesk()
{
    var cap = new AdCapPolicy();
    var ads = new MockAdsService(cap, () => DayKey, () => 0f);
    return new HospitalDoctors(cap, ads, () => DayKey);
}

static bool Contains(IReadOnlyList<string> funnel, string line)

```

```

    {
        for (int i = 0; i < funnel.Count; i++) if (funnel[i] == line) return true;
        return false;
    }

    static int CountFunnelTag(ReadOnlyList<string> funnel, string tag)
    {
        int n = 0;
        for (int i = 0; i < funnel.Count; i++)
        {
            int at = funnel[i].IndexOf('@');
            if (at > 0 && funnel[i].Substring(0, at) == tag) n++;
        }
        return n;
    }

    static void PassFail(string name, bool ok)
    {
        Lines.Add("[ " + (ok ? "PASS" : "FAIL") + " ] " + name);
        if (!ok) _failCount++;
    }

    static void WriteReport()
    {
        var sb = new StringBuilder();
        sb.AppendLine("G09 CrazyHospital AlbumCheck - " + System.DateTime.UtcNow.ToString("o"));
        foreach (var line in Lines) sb.AppendLine(line);
        sb.AppendLine("RESULT=" + (_failCount == 0 ? "ALL_GREEN" : "FAIL") + " fails=" + _failCount);
        string logsDir = Path.Combine(Path.GetFullPath(Path.Combine(Application.dataPath, "..")), "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(Path.Combine(logsDir, "g09_album_report.txt"), sb.ToString(), new UTF8Encoding(false));
    }
}

// ===== FILE: Assets\\Game\\Scripts\\Editor\\ArtCheck.cs =====
// ArtCheck.cs - the G09 T020 slice-c art gate (batchmode entry):
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyHospital.ArtCheck.Run -logFile <abs>
// G10 ArtCheck paradigm, G09-calibrated to the S2.5 arrival state (the 15
// stems are SHIPPED, so this gate is strict where G10 stayed lenient):
// Section1 closed_set_low - the 15 frozen sec7 stems derive from the
// live view contract (facade rank map + 7
// portraits + wangdali + event frame + kit)
// Section2 presence_low - every stem resolves a live catalog sprite
// whose name IS the stem (art-first fact)
// Section3 resolver_round_trip_low - the runtime facade resolver maps each
// rank onto the real art (the _r suffix key
// low; the C1081 silent-fallback bug class)
// Section4 null_safe_low - null/empty/missing lookups never throw
// Section5 fallback_face_low - every procedural face stays alive (X027)
// Section6 aqt_state_low - the AQT tier partition: 10 F-clean + 5

```

```

//                                     D-held (downgrade-pending re-rolls) covers
//                                     the whole closed set, machine-read face
// Exit codes: 0=all pass 1=fail/exception
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyHospital
{
    public static class ArtCheck
    {
        {
            const string DoctorWangdaliStem = "g09_doc_wangdali";    // held: A11R swap-in pending
            const int ExpectedTotal = 15;

            static readonly List<string> Lines = new List<string>();
            static int _passed;
            static int _failed;

            public static void Run()
            {
                int code;
                try
                {
                    Section1ClosedSetLaw();
                    Section2PresenceLaw();
                    Section3ResolverRoundTriplaw();
                    Section4NullSafeLaw();
                    Section5FallbackFaceLaw();
                    Section6AqtStateLaw();
                    code = _failed == 0 ? 0 : 1;
                }
                catch (Exception e)
                {
                    Lines.Add("CRASH - " + e.Message);
                    code = 1;
                }
                WriteReport();
                EditorApplication.Exit(code);
            }

            static string[] ClosedSet()
            {
                var stems = new List<string>();
                for (int r = 1; r <= 5; r++) stems.Add(HospitalArtView.FacadeStemOf(r));
                stems.AddRange(HospitalArtView.PortraitStems);
                stems.Add(DoctorWangdaliStem);
                stems.Add(HospitalArtView.EventFrameStem);
            }
        }
    }
}

```

```

        stems.Add(HospitalArtView.UikitStem);
        return stems.ToArray();
    }

    static void Section1ClosedSetLaw()
    {
        var stems = ClosedSet();
        var seen = new HashSet<string>();
        bool unique = true;
        foreach (var s in stems) if (!seen.Add(s)) unique = false;
        bool anchors = seen.Contains(HospitalArtView.FacadeStemOf(1))
            && seen.Contains(HospitalArtView.DirectorFemaleStem)
            && seen.Contains(HospitalArtView.DirectorMaleStem)
            && seen.Contains(HospitalArtView.EventFrameStem)
            && seen.Contains(HospitalArtView.UikitStem)
            && seen.Contains(DoctorWangdaliStem);
        PassFail("S1.closed_set_law",
            stems.Length == ExpectedTotal && unique && anchors,
            "stems=" + stems.Length + " (expected " + ExpectedTotal
            + ") unique=" + unique + " anchors=" + anchors
            + " (5 facades + 7 portraits + wangdali + event frame + kit)");
    }

    static void Section2PresenceLaw()
    {
        int present = 0, named = 0;
        foreach (var stem in ClosedSet())
        {
            var sprite = HospitalPlaceholder.ArtSprite(stem);
            if (sprite != null)
            {
                present++;
                if (sprite.name == stem) named++;
                else Lines.Add("FAIL detail - sprite name mismatch: " + stem + " -> " + sprite.name);
            }
            else Lines.Add("FAIL detail - stem missing from live catalog: " + stem);
        }
        PassFail("S2.presence_law", present == ExpectedTotal && named == ExpectedTotal,
            "catalog resolves all " + ExpectedTotal + " stems live, every sprite name == its stem ("
            + present + "/" + ExpectedTotal + " present, " + named + "/" + ExpectedTotal + " named)");
    }

    static void Section3ResolverRoundTriplaw()
    {
        // the runtime facade resolver must hand the view the REAL art for
        // every rank - a wrong key (bare _<rank> instead of _r<rank>)
        // silently falls back to the tint block (the C1081 bug class)
        int mapped = 0;
        for (int r = 1; r <= 5; r++)
        {

```

```

        var sprite = HospitalPlaceholder.Facade(r);
        if (sprite != null && sprite.name == HospitalArtView.FacadeStemOf(r)) mapped++;
        else lines.Add("FAIL detail - rank " + r + " resolver miss: "
            + (sprite == null ? "null" : sprite.name));
    }
    bool clampLow = HospitalPlaceholder.Facade(0) != null
        && HospitalPlaceholder.Facade(0).name == HospitalArtView.FacadeStemOf(1);
    bool clampHigh = HospitalPlaceholder.Facade(6) != null
        && HospitalPlaceholder.Facade(6).name == HospitalArtView.FacadeStemOf(1);
    PassFail("S3.resolver_round_trip_law", mapped == 5 && clampLow && clampHigh,
        "the resolver maps all 5 ranks onto real art with the R1 clamp law ("
            + mapped + "/5 mapped, clampLow=" + clampLow + ", clampHigh=" + clampHigh + ")");
}

static void Section4NullSafeLaw()
{
    bool noThrow = true;
    bool allNull = false;
    try
    {
        allNull = HospitalPlaceholder.ArtSprite(null) == null
            && HospitalPlaceholder.ArtSprite("") == null
            && HospitalPlaceholder.ArtSprite("g09_stem_missing") == null
            && HospitalPlaceholder.ArtPresent("g09_stem_missing") == false;
        var faceOnNull = HospitalPlaceholder.Face(null, Color.white, 8);
        var faceOnMissing = HospitalPlaceholder.Face("g09_stem_missing", Color.white, 8);
        noThrow = faceOnNull != null && faceOnMissing != null;
    }
    catch (Exception e) { noThrow = false; Lines.Add("FAIL detail - throw: " + e.Message); }
    PassFail("S4.null_safe_law", noThrow && allNull,
        "null/empty/missing lookups: null out, zero exceptions, fallback faces stay alive");
}

static void Section5FallbackFaceLaw()
{
    bool faces = true;
    string detail = "";
    try
    {
        var dept = HospitalPlaceholder.Dept();
        bool crossNamed = dept != null && dept.texture != null
            && dept.name.StartsWith("sy_green_cross", StringComparison.Ordinal);
        var center = dept.texture.GetPixel(dept.texture.width / 2, dept.texture.height / 2);
        bool crossDrawn = Mathf.Abs(center.r - HospitalPlaceholder.AccentMint.r) < 0.01f
            && Mathf.Abs(center.g - HospitalPlaceholder.AccentMint.g) < 0.01f;
        faces = HospitalPlaceholder.Patient() != null
            && HospitalPlaceholder.Doctor() != null
            && dept != null
            && HospitalPlaceholder.EventCard() != null
            && HospitalPlaceholder.Panel() != null
    }
}

```

```

        && crossNamed && crossDrawn;
        detail = "procedural family alive (patient/doctor/dept/event/panel)"
            + " + green-cross emblem center pixel = accent (red-cross avoidance face)";
    }
    catch (Exception e) { faces = false; detail = "throw: " + e.Message; }
    PassFail("S5.fallback_face_law", faces, detail);
}

static void Section6AqtStateLaw()
{
    // the C1079 verdict partition: 10 F-clean + 5 D-held (keyed-bg
    // residue / pre-swap-in states, honest downgrade records - the
    // G12 precedent-8 pattern) must cover the whole closed set
    var clean = new List<string>();
    for (int r = 1; r <= 5; r++) clean.Add(HospitalArtView.FacadeStemOf(r));
    clean.Add(HospitalArtView.DirectorMaleStem);
    clean.Add("g09_doc_beauty");
    clean.Add("g09_doc_moyu");
    clean.Add("g09_doc_threesec");
    clean.Add(HospitalArtView.UIKitStem);

    var held = new List<string>
    {
        HospitalArtView.DirectorFemaleStem,
        "g09_pat_hiccup",
        "g09_pat_restless",
        DoctorWangdaliStem,
        HospitalArtView.EventFrameStem,
    };

    var union = new HashSet<string>(clean);
    union.UnionWith(held);
    var closed = new HashSet<string>(ClosedSet());
    bool partition = clean.Count == 10 && held.Count == 5
        && union.SetEquals(closed);
    bool heldPresent = true;
    foreach (var stem in held)
        if (HospitalPlaceholder.ArtSprite(stem) == null) heldPresent = false;

    Lines.Add("AQT-D detail - F-clean x" + clean.Count + ": " + string.Join(", ", clean.ToArray()));
    Lines.Add("AQT-D detail - D-held x" + held.Count + " (downgrade-pending re-roll RES1-3/A11R/A14R): "
        + string.Join(", ", held.ToArray()));
    PassFail("S6.aqt_state_law", partition && heldPresent,
        "AQT tier partition F-clean=10 + D-held=5 covers the closed set (" + ExpectedTotal
        + "), every held stem stays present while awaiting swap-in");
}

static void PassFail(string name, bool ok, string detail)
{
    if (ok) _passed++; else _failed++;
}

```

```

        Lines.Add("[ " + (ok ? "PASS" : "FAIL") + " ] " + name + " - " + detail);
    }

    static void WriteReport()
    {
        var sb = new StringBuilder();
        sb.AppendLine("[ArtCheck] " + (_failed == 0 ? "ALL_GREEN" : "RED")
            + " passed=" + _passed + " failed=" + _failed
            + " at " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
        foreach (var line in Lines) sb.AppendLine(line);
        string dir = Path.Combine(Application.dataPath, "..", "Logs");
        if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
        File.WriteAllText(Path.Combine(dir, "g09_art_report.txt"), sb.ToString());
        Debug.Log(sb.ToString().Split('\n')[0]);
    }
}

// ===== FILE: Assets\_Game\Scripts\Editor\AudioCheck.cs =====
// AudioCheck.cs - the T021 wiring gate (batchmode entry):
// Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyHospital.AudioCheck.Run -logFile <abs>
// Section1: catalog_closed_set_law - the AudioSetup rebuild lands the
// delivered closed set (15 stems, the G09 audio manifest
// S1A1/A2/A3 batches) with live clips, unique keys, and
// nothing extra.
// Section2: stamp_map_law - every wired stamp resolves a catalog stem
// and the driven opening script fires no silent stamps
// (every fired stamp is wired, channel-start, or in the
// documented visual-only family).
// Section3: review_funnel_law - the rank engine's composed funnel seats
// resolve through the base-id law: pass -> evt_promote@ ->
// the promotion seal, fail -> evt_inspect@ -> the inspection
// whistle, band entry -> evt_band_change@rep_banner -> the
// banner-wall mount.
// Section4: named_cue_law - the three non-stamp face cues resolve stems
// and all three loop channels resolve.
// Section5: null_safe_law - unknown ids and missing stems skip
// silently (false/null, zero exceptions) on the headless
// host; the play positive face never throws.
// Section6: hook_family_law - the earworm hook family is wired (the
// ECG-beep + discharge-whistle hook baked into bgm_main plus
// the discharge whistle's own one-shot leg) and the delivered
// durations sit in the manifest bands.
// Exit codes: 0=all assertions pass 1=fail/exception
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

```

```

namespace Sy.CrazyHospital
{
    public static class AudioCheck
    {
        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        // the delivered closed set (G09 audio manifest, S1A1/A2/A3 batches)
        static readonly string[] ExpectedStems =
        {
            HospitalAudio.StemAmbLobby, HospitalAudio.StemAmbWard, HospitalAudio.StemBgmMain,
            HospitalAudio.StemSfxAlbum, HospitalAudio.StemSfxBanner, HospitalAudio.StemSfxBone,
            HospitalAudio.StemSfxCoin, HospitalAudio.StemSfxEmergency, HospitalAudio.StemSfxEvent,
            HospitalAudio.StemSfxInspect, HospitalAudio.StemSfxLaunch, HospitalAudio.StemSfxRank,
            HospitalAudio.StemSfxRegister, HospitalAudio.StemSfxStamp, HospitalAudio.StemSfxUpgrade,
        };

        // opening-script stamps that carry no audio by design (the
        // visual / number / narrative faces of the frozen script)
        static readonly HashSet<string> VisualOnlyStamps = new HashSet<string>
        {
            HospitalEventIds.FirstDecision, HospitalEventIds.FirstCoin,
            HospitalEventIds.FirstLaugh, HospitalEventIds.FirstPatientPortrait,
            HospitalEventIds.FirstDiagLine, HospitalEventIds.ReviewPreview,
            "coin_visual", "coin_number",
        };

        public static void Run()
        {
            int code;
            try
            {
                Section1CatalogClosedSetLaw();
                Section2StampMapLaw();
                Section3ReviewFunnellLaw();
                Section4NamedCueLaw();
                Section5NullSafeLaw();
                Section6HookFamilyLaw();
                code = _failed == 0 ? 0 : 1;
            }
            catch (Exception e)
            {
                Lines.Add("[AudioCheck] FAIL exception: " + e);
                _failed++;
                code = 1;
            }
            WriteReport();
            foreach (var line in Lines) Debug.Log(line);
            EditorApplication.Exit(code);
        }
    }
}

```

```

}

static void PassFail(string name, bool ok, string msg)
{
    if (ok) { _passed++; Lines.Add("PASS " + name + " - " + msg); }
    else { _failed++; Lines.Add("FAIL " + name + " - " + msg); }
}

static AudioClip Clip(string stem)
    => AudioRuntime.Catalog != null ? AudioRuntime.Catalog.Get(stem) : null;

static string SortedJoin(IEnumerable<string> xs)
{
    var list = new List<string>(xs);
    list.Sort(StringComparer.Ordinal);
    return string.Join(",", list);
}

static bool Contains(IReadOnlyList<string> hay, string needle)
{
    for (int i = 0; i < hay.Count; i++) if (hay[i] == needle) return true;
    return false;
}

static void Section1CatalogClosedSetLaw()
{
    int built = AudioSetup.Build();
    var catalog = Resources.Load<AudioCatalog>("AudioCatalog");
    PassFail("S1.catalog_rebuild", catalog != null && built == ExpectedStems.Length,
        "built=" + built + " expected=" + ExpectedStems.Length);

    var keys = new List<string>();
    int liveClips = 0;
    foreach (var e in catalog.entries)
    {
        if (e == null || string.IsNullOrEmpty(e.key)) continue;
        keys.Add(e.key);
        if (e.clip != null) liveClips++;
    }
    var expected = SortedJoin(ExpectedStems);
    var landed = SortedJoin(keys);
    PassFail("S1.closed_set", expected == landed,
        "entries=" + keys.Count + (expected == landed ? " exact-set" : " diff=" + landed));
    PassFail("S1.live_clips", keys.Count == liveClips && keys.Count == ExpectedStems.Length,
        "live=" + liveClips + "/" + keys.Count);

    var distinct = new HashSet<string>(keys, StringComparer.Ordinal);
    PassFail("S1.unique_keys", distinct.Count == keys.Count,
        "distinct=" + distinct.Count + " rows=" + keys.Count);
}

```

```

static void Section2StampMapLaw()
{
    int wired = 0, resolved = 0;
    foreach (var kv in HospitalAudio.Stamps)
    {
        wired++;
        if (Clip(kv.Value) != null) resolved++;
        else Lines.Add("FAIL detail - wired stem missing clip: " + kv.Key + " -> " + kv.Value);
    }
    PassFail("S2.wired_stamps_resolve", wired == resolved && wired > 0,
        "wired=" + wired + " resolved=" + resolved);

    // drive the frozen opening script: the hook owns t0, the
    // doors decision lands at T+4s, the queue head registers
    // half a second later with the coin on one tick, the first
    // laugh and the review preview land inside the 10s window
    var stage = HospitalStage.NewDean();
    stage.Advance(4.0);
    stage.OpenFirstDepartment(0);
    while (stage.SimTime < 12.0) stage.Advance(0.5);

    int silent = 0;
    foreach (var (id, t) in stage.Stamps)
    {
        if (HospitalAudio.StemForStamp(id) != null) continue;
        if (id == HospitalAudio.StampHospitalHook) continue; // the channel-start face
        if (VisualOnlyStamps.Contains(id)) continue; // documented no-audio family
        silent++;
        Lines.Add("FAIL detail - silent stamp: " + id + "@" + t);
    }
    PassFail("S2.no_silent_stamps", silent == 0, "fired=" + stage.Stamps.Count + " silent=" + silent);

    string[] beats =
    {
        HospitalAudio.StampHospitalHook, HospitalEventIds.DeptOpened,
        HospitalEventIds.FirstRegistration, HospitalAudio.CoinSfxStamp,
    };
    int fired = 0;
    foreach (var b in beats) if (stage.TimeOf(b) >= 0) fired++;
    PassFail("S2.wired_beats_fired", fired == beats.Length,
        "fired=" + fired + "/" + beats.Length + " registration@" +
        stage.TimeOf(HospitalEventIds.FirstRegistration).ToString("0.##"));
}

static void Section3ReviewFunnelLaw()
{
    // the pass settle: one slot, one cured patient, par reputation -
    // the R1 gate opens (the RankGateCheck SectionC recipe)
    var intake = new HospitalIntake(1);
    var events = new HospitalEvents(intake, 1);
}

```

```

var rank = new HospitalRank(intake, events, 1);
intake.OpenNextSlot(0);
intake.TryRegisterQueueHead();
intake.Advance(HospitalIntake.TreatmentSeconds + 0.5);
var pass = rank.RequestReview();
string promoteLine = null, inspectLine = null, bandLine = null;
foreach (var f in rank.Funnel)
{
    if (f.StartsWith(HospitalEventIds.Promote + "@", StringComparison.Ordinal)) promoteLine = f;
}
PassFail("S3.pass_settle_stem", pass.pass && promoteLine != null
    && HospitalAudio.StemForStamp(promoteLine) == HospitalAudio.StemSfxStamp
    && Clip(HospitalAudio.StemSfxStamp) != null,
    "funnel=" + promoteLine + " -> " + HospitalAudio.StemSfxStamp);

// the fail settle: zero slots = the hard track red - the
// rectification inspection face arms and the funnel books evt_inspect@
var intakeF = new HospitalIntake(1);
var eventsF = new HospitalEvents(intakeF, 1);
var rankF = new HospitalRank(intakeF, eventsF, 1);
var fail = rankF.RequestReview();
foreach (var f in rankF.Funnel)
{
    if (f.StartsWith(HospitalEventIds.Inspect + "@", StringComparison.Ordinal)) inspectLine = f;
}
PassFail("S3.fail_settle_stem", !fail.pass && inspectLine != null
    && HospitalAudio.StemForStamp(inspectLine) == HospitalAudio.StemSfxInspect
    && Clip(HospitalAudio.StemSfxInspect) != null,
    "funnel=" + inspectLine + " -> " + HospitalAudio.StemSfxInspect);

// the band entry: reputation pushed into the top band arms
// the banner wall - the composed seat resolves the banner stem
var intakeB = new HospitalIntake(1);
var rankB = new HospitalRank(intakeB, new HospitalEvents(intakeB, 1), 1);
intakeB.SetReputation(90); // rep_banner band (75-100)
rankB.Advance(1.0);
foreach (var f in rankB.Funnel)
{
    if (f.StartsWith(HospitalEventIds.BandChange + "@", StringComparison.Ordinal)) bandLine = f;
}
PassFail("S3.band_change_stem", bandLine != null
    && HospitalAudio.StemForStamp(bandLine) == HospitalAudio.StemSfxBanner
    && Clip(HospitalAudio.StemSfxBanner) != null,
    "funnel=" + bandLine + " -> " + HospitalAudio.StemSfxBanner);
}

static void Section4NamedCueLaw()
{
    int wired = 0, resolved = 0;
    foreach (var kv in HospitalAudio.Cues)

```

```

    {
        wired++;
        if (Clip(kv.Value) != null) resolved++;
        else Lines.Add("FAIL detail - cue stem missing clip: " + kv.Key + " -> " + kv.Value);
    }
    PassFail("S4.named_cues_resolve", wired == resolved && wired == 3,
        "cues=" + wired + " resolved=" + resolved);

    int channels = 0;
    foreach (var ch in HospitalAudio.Channels) if (Clip(ch) != null) channels++;
    PassFail("S4.loop_channels_resolve", channels == HospitalAudio.Channels.Length,
        "channels=" + channels + "/" + HospitalAudio.Channels.Length);
}

static void Section5NullSafeLaw()
{
    PassFail("S5.unknown_lookups_null",
        HospitalAudio.StemForStamp("bogus_stamp") == null
        && HospitalAudio.StemForStamp("evt_promote@r9:bogus_suffix") != null // base law survives composition
        && HospitalAudio.StemForCue("bogus_cue") == null
        && AudioRuntime.Get("g09_sfx_missing") == null,
        "unknown ids -> null; composed bases still resolve");

    bool noThrow = true;
    bool skipped = false;
    try
    {
        {
            skipped = HospitalAudio.Play("g09_sfx_missing") == false
                && HospitalAudio.PlayStamp("bogus_stamp") == false
                && HospitalAudio.PlayCue("bogus_cue") == false
                && HospitalAudio.Play(null) == false;
        }
    }
    catch (Exception e) { noThrow = false; Lines.Add("FAIL detail - throw: " + e.Message); }
    PassFail("S5.missing_skips_silently", noThrow && skipped,
        "missing stems/ids skip false, zero exceptions");

    bool playedFaceNoThrow = true;
    try
    {
        {
            HospitalAudio.Play(HospitalAudio.StemSfxCoin);
            HospitalAudio.StartMainFace();
            HospitalAudio.StartNightFace();
        }
    }
    catch (Exception e) { playedFaceNoThrow = false; Lines.Add("FAIL detail - play throw: " + e.Message); }
    PassFail("S5.play_positive_no_throw", playedFaceNoThrow,
        "real-stem play + channel starts never throw (device face out of gate scope)");
}

static void Section6HookFamilyLaw()
{

```

```

bool bgm = Clip(HospitalAudio.StemBgmMain) != null;
bool whistle = Clip(HospitalAudio.StemSfxLaunch) != null;
PassFail("S6.hook_family_wired", bgm && whistle,
    "bgm_baked_hook=" + bgm + " discharge_whistle=" + whistle);

// manifest duration bands: sfx 1-2s, amb 30s, bgm 60s
int inBand = 0, checkedSfx = 0;
foreach (var stem in ExpectedStems)
{
    var clip = Clip(stem);
    if (clip == null) continue;
    double len = clip.length;
    bool ok;
    if (stem == HospitalAudio.StemBgmMain) ok = len >= 50.0 && len <= 70.0;
    else if (stem == HospitalAudio.StemAmbLobby || stem == HospitalAudio.StemAmbWard) ok = len >= 25.0 && len <= 35.0;
    else { checkedSfx++; ok = len >= 0.5 && len <= 3.0; }
    if (ok) inBand++;
    else Lines.Add("FAIL detail - duration out of band: " + stem + " " + len.ToString("0.##") + "s");
}
PassFail("S6.duration_bands", inBand == ExpectedStems.Length,
    "in_band=" + inBand + "/" + ExpectedStems.Length + " sfx_checked=" + checkedSfx);
}

static void WriteReport()
{
    var sb = new StringBuilder();
    sb.AppendLine("[AudioCheck] " + (_failed == 0 ? "ALL_GREEN" : "RED")
        + " passed=" + _passed + " failed=" + _failed
        + " at " + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
    foreach (var line in Lines) sb.AppendLine(line);
    string dir = Path.Combine(Application.dataPath, "..", "Logs");
    if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
    File.WriteAllText(Path.Combine(dir, "g09_audio_report.txt"), sb.ToString());
    Debug.Log(sb.ToString().Split('\n')[0]);
}
}

// ===== FILE: Assets\_Game\Scripts\Editor\AudioSetup.cs =====
// AudioSetup.cs - the T021 catalog builder (the X027 ArtSetup paradigm
// applied to audio): scans Assets/_Game/Audio (BGM + SFX subfolders),
// resolves every imported AudioClip by filename stem, and (re)builds
// Assets/_Game/Resources/AudioCatalog.asset. Called by the AudioCheck
// gate before its assertions (self-healing catalog); also runnable
// standalone: Tuanjie.exe -batchmode -quit -executeMethod Sy.CrazyHospital.AudioSetup.Build
using System.Collections.Generic;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyHospital
{

```

```

public static class AudioSetup
{
    const string AudioRoot = "Assets/_Game/Audio";
    const string ResourcesFolder = "Assets/_Game/Resources";
    const string CatalogPath = "Assets/_Game/Resources/AudioCatalog.asset";

    /// <summary>(Re)build the audio catalog from the imported clips.
    /// Returns the entry count. Deterministic: entries sort by stem so
    /// two builds land the same key sequence (the parity face).</summary>
    public static int Build()
    {
        var guids = AssetDatabase.FindAssets("t:AudioClip", new[] { AudioRoot });
        var entries = new List<AudioCatalog.Entry>();
        foreach (var guid in guids)
        {
            var path = AssetDatabase.GUIDToAssetPath(guid);
            if (string.IsNullOrEmpty(path)) continue;
            var clip = AssetDatabase.LoadAssetAtPath<AudioClip>(path);
            if (clip == null) continue;
            entries.Add(new AudioCatalog.Entry { key = clip.name, clip = clip });
        }
        entries.Sort((a, b) => string.CompareOrdinal(a.key, b.key));

        if (!AssetDatabase.IsValidFolder(ResourcesFolder))
            AssetDatabase.CreateFolder("Assets/_Game", "Resources");
        var catalog = AssetDatabase.LoadAssetAtPath<AudioCatalog>(CatalogPath);
        if (catalog == null)
        {
            catalog = ScriptableObject.CreateInstance<AudioCatalog>();
            AssetDatabase.CreateAsset(catalog, CatalogPath);
        }
        catalog.entries = entries;
        EditorUtility.SetDirty(catalog);
        AssetDatabase.SaveAssets();
        AssetDatabase.Refresh();
        return entries.Count;
    }
}

// ===== FILE: Assets\_Game\Scripts\Editor\BootSequenceCheck.cs =====
using System.IO;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;

namespace Sy.CrazyHospital
{
    /// <summary>
    /// Seven-step boot dry-run (batchmode entry):

```

```

/// Tuanjie.exe -batchmode -quit -executeMethod Sy.CrazyHospital.BootSequenceCheck.Run -logFile <abs>
/// Exit codes: 0=all steps pass and well formed 1=fail/exception 3=scenes missing (run ProjectSetup first)
/// </summary>
public static class BootSequenceCheck
{
    public static void Run()
    {
        int code;
        try
        {
            string mainAsset = "Assets/_Game/Scenes/S09_Main.unity";
            string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
            if (!File.Exists(Path.Combine(projectRoot, mainAsset)))
            {
                Debug.LogError("[BootSequenceCheck] main scene missing - run ProjectSetup first");
                EditorApplication.Exit(3);
                return;
            }

            Boot.BootServices.RegisterAll();
            bool switched = false;
            var sequence = new Boot.BootSequence(delegate
            {
                EditorSceneManager.OpenScene(mainAsset, OpenSceneMode.Single);
                switched = true;
                return true;
            });
            bool allOk = sequence.RunAll();

            var sb = new StringBuilder();
            foreach (var line in sequence.Lines) sb.AppendLine(line);
            var logsDir = Path.Combine(projectRoot, "Logs");
            if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
            File.WriteAllText(Path.Combine(logsDir, "boot_sequence_report.txt"), sb.ToString(), new UTF8Encoding(false));

            bool wellFormed = sequence.Lines.Count == Boot.BootSequence.TotalSteps && switched;
            for (int i = 0; i < sequence.Lines.Count; i++)
            {
                Debug.Log(sequence.Lines[i]);
                if (!sequence.Lines[i].StartsWith("[BOOT] " + (i + 1) + "/" + Boot.BootSequence.TotalSteps + " "))
                    wellFormed = false;
            }

            code = allOk && wellFormed ? 0 : 1;
        }
        catch (System.Exception e)
        {
            Debug.LogError("[BootSequenceCheck] exception: " + e);
            code = 1;
        }
    }
}

```

```

        EditorApplication.Exit(code);
    }
}

// ===== FILE: Assets\_Game\Scripts\Editor\BuildService.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyHospital
{
    /// <summary>
    /// Compile zero-warning gate stub (batchmode entry).
    /// Usage: Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyHospital.BuildService.CompileCheck -logFile <abs>
    /// Exit codes: 0=zero error zero warning 1=has error 2=has warning 3=no -logFile 4=exception
    /// </summary>
    public static class BuildService
    {
        public const int Ok = 0;
        public const int HasError = 1;
        public const int HasWarning = 2;
        public const int NoLogFile = 3;
        public const int Exception = 4;

        public static void CompileCheck()
        {
            var report = new CompileCheckReport
            {
                project = "CrazyHospital",
                unityVersion = Application.unityVersion,
                atUtc = DateTime.UtcNow.ToString("o"),
                errors = new List<string>(),
                warnings = new List<string>(),
            };

            int code = Ok;
            try
            {
                string logPath = FindArgValue("-logFile");
                if (string.IsNullOrEmpty(logPath) || !File.Exists(logPath))
                {
                    report.status = "no_log_file";
                    code = NoLogFile;
                }
            }
            else
            {
                ScanLog(logPath, report);
            }
        }
    }
}

```

```

        if (report.errors.Count > 0) { report.status = "error"; code = HasError; }
        else if (report.warnings.Count > 0) { report.status = "warning"; code = HasWarning; }
        else { report.status = "ok"; code = 0k; }
    }
}
catch (Exception e)
{
    report.status = "exception";
    report.exception = e.Message + "\n" + e.StackTrace;
    code = Exception;
}

WriteReport(report);
EditorApplication.Exit(code);
}

/// <summary>Reads a command line value, supporting both "-logFile x" and "-logFile=x".</summary>
internal static string FindArgValue(string name)
{
    var args = Environment.GetCommandLineArgs();
    for (int i = 0; i < args.Length; i++)
    {
        if (string.Equals(args[i], name, StringComparison.OrdinalIgnoreCase))
        {
            if (i + 1 < args.Length) return args[i + 1];
        }
        else if (args[i].StartsWith(name + "=", StringComparison.OrdinalIgnoreCase))
        {
            return args[i].Substring(name.Length + 1);
        }
    }
    return null;
}

/// <summary>Counts compile errors/warnings (in batchmode compilation happens before executeMethod).</summary>
static void ScanLog(string logPath, CompileCheckReport report)
{
    using var fs = new FileStream(logPath, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
    using var sr = new StreamReader(fs, Encoding.UTF8);
    var seenErrors = new HashSet<string>();
    var seenWarnings = new HashSet<string>();
    string line;
    while ((line = sr.ReadLine()) != null)
    {
        if (line.IndexOf(": error CS", StringComparison.Ordinal) >= 0
            || line.IndexOf("Scripts have compiler errors", StringComparison.Ordinal) >= 0
            || line.IndexOf("Error building Player", StringComparison.Ordinal) >= 0)
        {
            if (seenErrors.Add(Trim(line))) report.errors.Add(Trim(line));
        }
    }
}

```

```

        else if (line.IndexOf(": warning CS", StringComparison.Ordinal) >= 0)
        {
            if (seenWarnings.Add(Trim(line))) report.warnings.Add(Trim(line));
        }
    }
    report.errorCount = report.errors.Count;
    report.warningCount = report.warnings.Count;
}

static string Trim(string s)
{
    s = s.TrimEnd();
    return s.Length > 500 ? s.Substring(0, 500) + "... " : s;
}

static void WriteReport(CompileCheckReport report)
{
    try
    {
        var projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
        var logsDir = Path.Combine(projectRoot, "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(
            Path.Combine(logsDir, "compile_report.json"),
            JsonUtility.ToJson(report, true),
            new UTF8Encoding(false));
        Console.WriteLine($"[CompileCheck] status={report.status} errors={report.errorCount} warnings={report.warningCount} -
> Logs/compile_report.json");
    }
    catch (Exception e)
    {
        Console.WriteLine("[CompileCheck] report write failed: " + e.Message);
    }
}

/// <summary>
/// T012 build face: WeChat minigame package build (batchmode).
/// Ported from Sy.CrazyStocks.BuildService.BuildWeixin (G10 C246
/// milestone-proven lineage, X123 law): raw-container format pin (no
/// public API face for the weixinMiniGame* keys, drive the serialized
/// key, restore in finally) and the SY_WECHAT bridge define
/// (com.sy.platform WxPlatformSuite only compiles under it). G09 has
/// no grey-cohort arm, so no SY_SUBMISSION define is injected.
/// Exit 0 = build succeeded, 1 = switch/build failed, 4 = exception.
/// Usage: Tuanjie.exe -batchmode -quit -projectPath &lt;p&gt; -executeMethod Sy.CrazyHospital.BuildService.BuildWeixin -
logFile &lt;abs&gt;
/// </summary>
public static void BuildWeixin()
{
    int code;
    SerializedObject psObj = null;
    int fmtPrev = 0;

```

```

bool pinned = false;
try
{
    var scenes = EditorBuildSettings.scenes;
    if (scenes == null || scenes.Length == 0)
    {
        Console.WriteLine("[BuildWeixin] no scenes pinned in EditorBuildSettings");
        EditorApplication.Exit(1);
        return;
    }
    var levellist = new List<string>();
    foreach (var s in scenes)
        if (s.enabled) levellist.Add(s.path);
    if (levellist.Count == 0)
    {
        Console.WriteLine("[BuildWeixin] no enabled scenes");
        EditorApplication.Exit(1);
        return;
    }

    bool switched = EditorUserBuildSettings.SwitchActiveBuildTarget(
        BuildTargetGroup.WeixinMiniGame, BuildTarget.WeixinMiniGame);
    Console.WriteLine("[BuildWeixin] switchActiveTarget=" + switched
        + " scenes=" + levellist.Count);
    if (!switched) { EditorApplication.Exit(1); return; }

    string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
    string outDir = Path.Combine(projectRoot, "MiniGame", "weixin", "build");
    Directory.CreateDirectory(outDir);

    // X123: pin the raw-container format before building (brotli
    // output would break the later 4MB main-package split contract).
    var psAll = UnityEngine.Resources.FindObjectsOfTypeAll(typeof(PlayerSettings));
    if (psAll == null || psAll.Length == 0)
    {
        Console.WriteLine("[BuildWeixin] X123 abort: PlayerSettings instance not found");
        EditorApplication.Exit(1);
        return;
    }
    psObj = new SerializedObject(psAll[0]);
    var fmtProp = psObj.FindProperty("weixinMiniGameCompressionFormat");
    if (fmtProp == null)
    {
        Console.WriteLine("[BuildWeixin] X123 abort: serialized key weixinMiniGameCompressionFormat missing");
        EditorApplication.Exit(1);
        return;
    }
    fmtPrev = fmtProp.intValue;
    fmtProp.intValue = 0;
    psObj.ApplyModifiedProperties();
}

```

```

Console.WriteLine("[BuildWeixin] X123 fmtPin prev=" + fmtPrev
    + " set=0 readback=" + fmtProp.intValue);
if (fmtProp.intValue != 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: pin readback mismatch - no build");
    EditorApplication.Exit(1);
    return;
}
pinned = true;

string[] buildDefines = { "SY_WECHAT" };

var report = BuildPipeline.BuildPlayer(new BuildPlayerOptions
{
    scenes = levelList.ToArray(),
    locationPathName = outDir,
    target = BuildTarget.WeixinMiniGame,
    options = BuildOptions.None,
    extraScriptingDefines = buildDefines,
});

string result = report.summary.result.ToString();
long totalBytes = (long)report.summary.totalSize;
Console.WriteLine("[BuildWeixin] result=" + result
    + " totalSizeBytes=" + totalBytes
    + " errors=" + report.summary.totalErrors
    + " warnings=" + report.summary.totalWarnings);

try
{
    var logsDir = Path.Combine(projectRoot, "Logs");
    if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
    File.WriteAllText(
        Path.Combine(logsDir, "build_weixin_report.json"),
        "{ \"result\": \"" + result + "\", \"totalSizeBytes\": " + totalBytes
        + ", \"errors\": " + report.summary.totalErrors
        + ", \"warnings\": " + report.summary.totalWarnings
        + ", \"defineInjected\": \"" + string.Join(";", buildDefines) + "\" "
        + ", \"atUtc\": \"" + DateTime.UtcNow.ToString("o") + "\" }",
        new UTF8Encoding(false));
}
catch (Exception e) { Console.WriteLine("[BuildWeixin] report write failed: " + e.Message); }

code = report.summary.result == UnityEditor.Build.Reporting.BuildResult.Succeeded ? 0 : 1;
}
catch (Exception e)
{
    Console.WriteLine("[BuildWeixin] exception: " + e.Message + " | " + e.StackTrace);
    code = 1;
}

```

```

        finally
        {
            if (pinned && psObj != null)
            {
                try
                {
                    psObj.Update();
                    var rp = psObj.FindProperty("weixinMiniGameCompressionFormat");
                    if (rp != null)
                    {
                        rp.intValue = fmtPrev;
                        psObj.ApplyModifiedProperties();
                        Console.WriteLine("[BuildWeixin] X123 fmtRestored=" + rp.intValue);
                    }
                    else
                    {
                        Console.WriteLine("[BuildWeixin] X123 WARNING: restore key missing");
                    }
                }
                catch (Exception rex)
                {
                    Console.WriteLine("[BuildWeixin] X123 restore failed: " + rex.Message);
                }
            }
        }
        EditorApplication.Exit(code);
    }
}

[Serializable]
public class CompileCheckReport
{
    public string project;
    public string unityVersion;
    public string atUtc;
    public string status;
    public int errorCount;
    public int warningCount;
    public List<string> errors;
    public List<string> warnings;
    public string exception;
}

}

// ===== FILE: Assets\Game\Scripts\Editor\CheckBundle.cs =====
// CheckBundle.cs - main-package budget gate (G09-T015, S2 face, U037 face ownership).
// Ported from Sy.G12 G12CheckBundle (C216 paradigm, G06 S2.5-T003 lineage): BuildPackage
// asserts the 20MB total from the build report, this gate owns the 4MB main-package
// split line. Measures the built weixin artifact dir (MiniGame/weixin/build/Build,
// X123 raw-container form, same outDir contract as BuildService.BuildWeixin) and asserts:
// main face = build.loader.js* + build.framework.js.* + build.wasm.*

```

```

//          (engine runtime that must ride the weixin main package) <= 4MB
//  data face  = build.data.* (subpackage candidate)
//  total face = main + data <= 20MB
// Face division mirrors the monorepo tools/CheckBundle.ps1 semantics (one truth,
// no second dialect). Anti-hallucination: missing dir/pieces or zero-byte faces
// can never pass (exit 3/1), evidence is the machine-readable report only.
// Usage: Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.CrazyHospital.CheckBundle.Run -logFile <abs>
// Exit codes: 0=pass 1=main over 4MB (or zero-byte evidence) 2=total over 20MB 3=missing inputs 4=exception
// Report: Logs/checkbundle_report.json
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

namespace Sy.CrazyHospital
{
    public static class CheckBundle
    {
        {
            public const long MainLimitBytes = 4L * 1024 * 1024; // 4MB weixin main-package red line
            public const long TotallimitBytes = 20L * 1024 * 1024; // 20MB whole-package red line

            public static void Run()
            {
                var report = new BundleReport
                {
                    project = "CrazyHospital",
                    atUtc = DateTime.UtcNow.ToString("o"),
                    mainFiles = new List<string>(),
                    dataFiles = new List<string>(),
                    otherFiles = new List<string>(),
                };
                int code;
                try
                {
                    code = Measure(report);
                }
                catch (Exception e)
                {
                    report.status = "exception";
                    report.exception = e.Message + "\n" + e.StackTrace;
                    code = 4;
                }
                WriteReport(report);
                Console.WriteLine("[G09CheckBundle] status=" + report.status
                    + " mainPkgBytes=" + report.mainPkgBytes + "/" + MainLimitBytes
                    + " totalFaceBytes=" + report.totalFaceBytes + "/" + TotallimitBytes
                    + " coverage=" + report.coverageComplete
                    + " -> Logs/checkbundle_report.json");
            }
        }
    }
}

```

```

    EditorApplication.Exit(code);
}

static int Measure(BundleReport report)
{
    string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
    string artifactDir = Path.Combine(projectRoot, "MiniGame", "weixin", "build", "Build");
    if (!Directory.Exists(artifactDir))
    {
        report.status = "missing_package";
        report.exception = "artifact dir not found: " + artifactDir
            + " (build first: Sy.CrazyHospital.BuildService.BuildWeixin)";
        return 3;
    }

    long mainBytes = 0, dataBytes = 0, otherBytes = 0;
    bool hasLoader = false, hasFramework = false, hasWasm = false, hasData = false;
    foreach (var f in Directory.GetFiles(artifactDir))
    {
        string name = Path.GetFileName(f);
        long len = new FileInfo(f).Length;
        if (name.StartsWith("build.loader", StringComparison.Ordinal))
        { mainBytes += len; hasLoader = true; report.mainFiles.Add(name + ":" + len); }
        else if (name.StartsWith("build.framework", StringComparison.Ordinal))
        { mainBytes += len; hasFramework = true; report.mainFiles.Add(name + ":" + len); }
        else if (name.StartsWith("build.wasm", StringComparison.Ordinal))
        { mainBytes += len; hasWasm = true; report.mainFiles.Add(name + ":" + len); }
        else if (name.StartsWith("build.data", StringComparison.Ordinal))
        { dataBytes += len; hasData = true; report.dataFiles.Add(name + ":" + len); }
        else
        { otherBytes += len; report.otherFiles.Add(name + ":" + len); }
    }

    report.mainPkgBytes = mainBytes;
    report.dataFaceBytes = dataBytes;
    report.otherFaceBytes = otherBytes;
    report.totalFaceBytes = mainBytes + dataBytes + otherBytes;
    report.coverageComplete = hasLoader && hasFramework && hasWasm && hasData;

    // Anti-hallucination: piece coverage + non-zero faces are hard evidence gates.
    if (!report.coverageComplete || mainBytes <= 0 || dataBytes <= 0)
    {
        report.status = "missing_inputs";
        report.exception = "coverage incomplete (loader=" + hasLoader
            + " framework=" + hasFramework + " wasm=" + hasWasm + " data=" + hasData
            + ") or zero-byte face (main=" + mainBytes + " data=" + dataBytes + ")";
        return 3;
    }
}
if (mainBytes > MainLimitBytes)
{

```

```

        report.status = "main_over";
        report.exception = "main package " + mainBytes + "B > " + MainLimitBytes + "B (split contract broken)";
        return 1;
    }
    if (report.totalFaceBytes > TotalLimitBytes)
    {
        report.status = "total_over";
        report.exception = "total face " + report.totalFaceBytes + "B > " + TotalLimitBytes + "B";
        return 2;
    }
    report.status = "ok";
    report.mainHeadroomBytes = MainLimitBytes - mainBytes;
    return 0;
}

static void WriteReport(BundleReport report)
{
    try
    {
        var projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
        var logsDir = Path.Combine(projectRoot, "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(
            Path.Combine(logsDir, "checkbundle_report.json"),
            JsonUtility.ToJson(report, true),
            new UTF8Encoding(false));
    }
    catch (Exception e)
    {
        Console.WriteLine("[G09CheckBundle] report write failed: " + e.Message);
    }
}

}

[Serializable]
public class BundleReport
{
    public string project;
    public string atUtc;
    public string status;
    public long mainPkgBytes;
    public long mainHeadroomBytes;
    public long dataFaceBytes;
    public long otherFaceBytes;
    public long totalFaceBytes;
    public bool coverageComplete;
    public List<string> mainFiles;
    public List<string> dataFiles;
    public List<string> otherFiles;
    public string exception;
}

```

(middle pages omitted)

pages 31 - 155 of 185 omitted per first-30 + last-30 excerpt rule.

```

double remaining = _simSeconds;
while (remaining > 0.0000001)
{
    double step = Math.Min(WalkSliceSeconds, remaining);
    _stage.Advance(step);
    remaining -= step;
    // the night desk: a freed bed takes the next companion
    if (intake.OpenSlots > 0 && intake.QueueCount > 0)
        intake.TryRegisterQueueHead();
}

_earnHc = intake.GoldHc - _goldBeforeHc;
_armed = _earnHc > 0; // one offer per span, only when there is something to double
BuildSummary();
}

void BuildSummary()
{
    var intake = _stage.Intake;
    _summary.Add(HospitalTexts.EnText(HospitalTexts.KeyNightTide) + " " + _tideArrivals
        + " EXTRA WALK-INS - THE HALL SLEPT FULL OF COMPANIONS");
    _summary.Add("away " + HoursStr(_spanSeconds) + "h -> " + HoursStr(_simSeconds) + "h at "
        + HospitalData.OfflineRatePercent + "% pace");
    int rolled = _stage.Events.RollCount - _eventsRolledBefore;
    if (rolled > 0 || _stage.Events.PendingCard != null)
        _summary.Add("the night desk kept busy: " + rolled + " event cards rolled, "
            + (_stage.Events.PendingCard != null ? "one is waiting on your desk" : "none left waiting"));
    _summary.Add("the street never emptied: " + intake.QueueCount + " still lining up outside");
    _summary.Add("gold " + HcStr(_goldBeforeHc) + " -> " + HcStr(intake.GoldHc)
        + " (" + (_earnHc >= 0 ? "+" : "") + HcStr(_earnHc) + ")");
}

/// <summary>The double offer claim (the shared double category,
/// one offer per span): the armed guard first (a dead tap changes
/// nothing - zero state, zero budget), the category precheck
/// second, the rewarded view third; only the completed view books
/// the earned gold a second time through the event ledger line.
/// A capped tap refuses BEFORE the ad; an aborted view keeps the
/// span armed for the retry.</summary>
public async Task<bool> ClaimDouble()
{
    if (!_armed) return false; // the dead tap: zero state, zero budget
    if (_adCap == null || _ads == null) return false;
    if (!_adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(DoublePoint)))
    {
        _funnel.Add("evt_ad_offer_capped@" + SpanTag);
        return false; // the capped tap refuses BEFORE the ad ever plays
    }
    _funnel.Add("evt_ad_offer@" + SpanTag);
    RewardResult r = await _ads.ShowRewarded(DoublePoint);

```

```

        if (r == null || !r.Completed) return false; // the aborted view: armed stays, the retry window stays open
        _stage.Intake.BookEventGoldHc(_earnHc); // the x2 injection rides the event ledger line - the wallet law holds by
construction
        _funnel.Add("evt_offline_double@" + SpanTag + ":" + _earnHc);
        _armed = false;
        _claimed = true;
        return true;
    }

    /// <summary>evt_ad funnel (the registered-service lifecycle face):
    /// show/complete/fail of the double point land in order.</summary>
    void OnAdLifecycle(AdLifecycle e)
    {
        if (e.PointId != DoublePoint) return;
        _funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + SpanTag);
    }

    static string HoursStr(double seconds)
        => (seconds / 3600.0).ToString("0.##", CultureInfo.InvariantCulture);

    static string HcStr(long hc)
        => (hc / 100.0).ToString("0.##", CultureInfo.InvariantCulture);

    /// <summary>Canonical fingerprint - the zero-RNG parity instrument
    /// (the settle state plus the whole stage opening signature).</summary>
    public string Fingerprint()
    {
        var intake = _stage.Intake;
        var sb = new StringBuilder();
        sb.Append("span").Append(_spanIndex).Append('@')
            .Append(_spanSeconds.ToString("0.##", CultureInfo.InvariantCulture)).Append('>')
            .Append(_simSeconds.ToString("0.##", CultureInfo.InvariantCulture));
        sb.Append("|tide").Append(_tideArrivals);
        sb.Append("|earn").Append(_earnHc);
        sb.Append("|armed").Append(_armed ? 1 : 0).Append(_claimed ? "C" : "-");
        sb.Append("|queue").Append(intake.QueueCount).Append(',').Append(intake.ArrivalTotal)
            .Append(',').Append(intake.PatientTotal).Append(',').Append(intake.CuredTotal);
        sb.Append("|fun").Append(_funnel.Count).Append("|sum").Append(_summary.Count);
        foreach (var s in _summary) sb.Append('#').Append(s);
        sb.Append("||").Append(_stage.OpeningSig());
        return sb.ToString();
    }
}

}

// ===== FILE: Assets\\Game\\Scripts\\Runtime\\Game\\HospitalPlaceholder.cs =====
// HospitalPlaceholder.cs - procedural placeholder faces for Crazy Hospital
// (G09-T004). Art-first iron rule (X027): when ArtRuntime holds the stem the
// real sprite wins, otherwise the face falls back to a procedural render -
// views render either way and no headless gate ever depends on art presence.
// The series accent is mint teal #6ED8C8 (the G09 brand face). The emblem
// family is the GREEN CROSS (the red-cross avoidance law: a fictional green

```

```

// cross marks every medical face, never a real emblem).
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;

namespace Sy.CrazyHospital
{
    public static class HospitalPlaceholder
    {
        /// <summary>Series accent: mint teal #6ED8C8 (design-language face).</summary>
        public static readonly Color AccentMint = new Color(110f / 255f, 216f / 255f, 200f / 255f);

        // ---- rank tints: candy clinic -> cream -> fresh mint -> clinical
        // teal -> sterile steel (the vertical facade warms as the hospital
        // ranks up) ----
        static readonly Color[] RankTints =
        {
            new Color(0.96f, 0.86f, 0.80f), // R1 candy clinic
            new Color(0.97f, 0.94f, 0.82f), // R2 warm cream
            new Color(0.78f, 0.94f, 0.88f), // R3 fresh mint
            new Color(0.62f, 0.82f, 0.86f), // R4 clinical teal
            new Color(0.52f, 0.62f, 0.72f), // R5 sterile steel
        };

        // ---- stems (art ships under these keys in S2.5; today they miss -> fallback) ----
        public const string FacadeStem = "g09_facade";
        public const string PatientStem = "g09_patient";
        public const string DoctorStem = "g09_doctor";
        public const string DeptStem = "g09_dept";
        public const string EventCardStem = "g09_eventcard";
        public const string PanelStem = "g09_panel";

        static readonly Dictionary<string, Sprite> _cache = new Dictionary<string, Sprite>();

        /// <summary>True when real art ships under the stem (S2.5 face - the
        /// placeholder gates branch on this so art arrival never breaks them).</summary>
        public static bool ArtPresent(string stem) => ArtRuntime.Get(stem) != null;

        /// <summary>The shipped art sprite for a stem (null on the placeholder
        /// path) - editor gates compare against this without touching Sy.Core.</summary>
        public static Sprite ArtSprite(string stem) => ArtRuntime.Get(stem);

        /// <summary>Hospital facade tinted by rank (1-5, out-of-range folds
        /// to the R1 candy face). The vertical growth roll face. Key law: the
        /// shipped catalog stems carry the _r rank suffix (art book sec7,
        /// g09_facade_r1..r5) - a bare _&lt;rank&gt; probe silently misses the
        /// art and falls back to the tint block (the C1081 wiring bug this
        /// resolver must never reintroduce; ArtCheck S3 owns the law).</summary>
        public static Sprite Facade(int rank)
        {

```

```

    int i = rank - 1;
    if (i < 0 || i >= RankTints.Length) i = 0;
    return Face(FacadeStem + "_r" + (i + 1), RankTints[i], 64);
}

public static Sprite Patient() => Face(PatientStem, new Color(0.92f, 0.80f, 0.88f), 32);
public static Sprite Doctor() => Face(DoctorStem, new Color(0.80f, 0.88f, 0.95f), 32);

/// <summary>The department face: art-first, else a green-cross block
/// (the fictional emblem, red-cross avoidance law).</summary>
public static Sprite Dept()
{
    Sprite art = ArtRuntime.Get(DeptStem);
    if (art != null) return art;
    Sprite cached;
    if (_cache.TryGetValue(DeptStem, out cached) && cached != null) return cached;
    Sprite made = ProceduralCross(32, AccentMint, new Color(0.94f, 0.98f, 0.97f));
    _cache[DeptStem] = made;
    return made;
}

public static Sprite EventCard() => Face(EventCardStem, new Color(0.95f, 0.90f, 0.66f), 48);
public static Sprite Panel() => Face(PanelStem, new Color(0.14f, 0.18f, 0.20f, 0.92f), 64);

/// <summary>Art-first with null-safe procedural fallback (X027 iron rule).</summary>
public static Sprite Face(string stem, Color color, int px)
{
    Sprite art = ArtRuntime.Get(stem);
    if (art != null) return art;
    string key = stem + "#" + px;
    Sprite cached;
    if (_cache.TryGetValue(key, out cached) && cached != null) return cached;
    Sprite made = ProceduralSprite(color, px);
    _cache[key] = made;
    return made;
}

/// <summary>Procedural color block: an uncompressed readable px*px square.</summary>
public static Sprite ProceduralSprite(Color color, int px)
{
    var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
    var pixels = new Color[px * px];
    for (int i = 0; i < pixels.Length; i++) pixels[i] = color;
    tex.SetPixels(pixels);
    tex.Apply(false, false);
    tex.name = "sy_procedural_" + px;
    var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
    sprite.name = tex.name;
    return sprite;
}

```

```

/// <summary>The green-cross procedural emblem: a plus-shaped cross in
/// the accent color on a soft block (the fictional medical mark - a
/// pure pixel draw, stable forever for gate pixel reads).</summary>
public static Sprite ProceduralCross(int px, Color cross, Color background)
{
    var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
    var pixels = new Color[px * px];
    int arm = px / 8; // arm thickness (>=1 on any px >= 8)
    if (arm < 1) arm = 1;
    int mid = px / 2;
    for (int y = 0; y < px; y++)
    {
        for (int x = 0; x < px; x++)
        {
            bool inCross = (x >= mid - arm && x <= mid + arm - 1) || (y >= mid - arm && y <= mid + arm - 1);
            pixels[y * px + x] = inCross ? cross : background;
        }
    }
    tex.SetPixels(pixels);
    tex.Apply(false, false);
    tex.name = "sy_green_cross_" + px;
    var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
    sprite.name = tex.name;
    return sprite;
}

/// <summary>The rank tint for a hospital rank (1-5) - the visual
/// progress face for checks.</summary>
public static Color TintForRank(int rank)
{
    int i = rank - 1;
    if (i < 0 || i >= RankTints.Length) i = 0;
    return RankTints[i];
}
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\HospitalRank.cs =====
// HospitalRank.cs - the rank-review gate engine (G09-T008, SYS05/SYS06).
// The five-rank ladder R1-R5 walks four dual-track gates (numeric pack
// section three): the hard metrics (departments / cure rate / reputation)
// AND the inspection score must hold together - a promotion is a new
// gameplay ticket, never a wall. A failed review only arms the
// rectification comedy (the review expert rides the catapult into the
// hospital's own ward) and re-arms the next review window at the frozen
// inspection cadence (180s, the events-engine cycle - zero new numbers).
// The settlement face is the frozen closed set: pass swings reputation
// +3, fail -3 (HospitalData.ReviewRepSwing); the E01 review bonus folds
// +3 into the score, E04 folds +2 (the events engine already books
// both). The reputation band is the second numeric line (section four):

```

```

// band changes book their visible face flags (banner growth at the top
// band, bad-review comedy at the bottom) plus the evt_band_change
// funnel line - the street flow itself already rides the frozen
// reputation function (TrafficAt), so the band face never re-models it.
// The engine keeps its OWN clock and funnel; the opening signature of
// the stage stays byte-identical until a review resolves (identity
// law). A resolved promotion folds the swap into both hosts inside the
// settle: the intake economy row (PromoteToRank) and the events stage
// pool (SetRank) - one verdict, three faces agree (the T008 host
// wiring). Pure C#, no MonoBehaviour.
using System;
using System.Collections.Generic;

namespace Sy.CrazyHospital
{
    /// <summary>One review settlement (the dual-track verdict).</summary>
    public struct ReviewVerdict
    {
        public bool pass;
        public bool hardOk;
        public bool scoreOk;
    }

    /// <summary>SYS05/SYS06: the rank-review gate + reputation band
    /// engine. The host wires Advance(dt); headless gates drive the
    /// static solver and the gate faces directly.</summary>
    public sealed class HospitalRank
    {
        // ---- the frozen review laws (closed set, zero new numbers) ----
        public const double ReviewCooldownSeconds = HospitalEvents.CadenceSeconds; // re-review = the next inspection cycle
        public const int ScoreBonus = HospitalData.ReviewRepSwing; // the E01 pass folds +3 into the score

        readonly HospitalIntake _intake;
        readonly HospitalEvents _events;
        readonly List<string> _funnel = new List<string>();
        double _t;
        double _cooldownUntil = -1; // sim time until the next review window opens (-1 = open)
        int _rank = 1;
        int _promotions;
        int _failedReviews;
        int _lastVerdictRank; // rank the last review ran under (0 = never)
        bool _lastPass;
        bool _ceremonyArmed; // pass -> the stamp ceremony (fireworks + banner rain)
        bool _remediationArmed; // fail -> the rectification comedy face
        bool _badReviewComedyArmed; // band entry into rep_bad
        bool _bannerGrowthArmed; // band entry into rep_banner
        string _bandId = "";

        public HospitalRank(HospitalIntake intake, HospitalEvents events, int startRank = 1)
        {

```

```

        _intake = intake;
        _events = events;
        _rank = startRank < 1 ? 1 : startRank;
        _bandId = BandFace().id;
    }

    // ---- live faces ----
    public double SimTime => _t;
    public int Rank => _rank;
    public int Promotions => _promotions;
    public int FailedReviews => _failedReviews;
    public int LastVerdictRank => _lastVerdictRank;
    public bool LastPass => _lastPass;
    public bool CeremonyArmed => _ceremonyArmed;
    public bool RemediationArmed => _remediationArmed;
    public bool BadReviewComedyArmed => _badReviewComedyArmed;
    public bool BannerGrowthArmed => _bannerGrowthArmed;
    public string BandId => _bandId;
    public IReadOnlyList<string> Funnel => _funnel;
    public bool ReviewWindowOpen => _t >= _cooldownUntil;

    /// <summary>The gate the current rank faces (null at R5: the
    /// ladder is complete, the top-grade stamp is the ending face).</summary>
    public ReviewGateDef CurrentGate
    {
        get
        {
            foreach (var g in HospitalData.Gates())
                if (g.fromRank == _rank) return g;
            return null;
        }
    }

    /// <summary>The live inspection score: the events-engine delta
    /// plus the armed E01 review bonus (the closed set: E04 +-2,
    /// E01 pass +3).</summary>
    public int InspectScore
        => (_events != null ? _events.InspectScoreDelta : 0)
        + (_events != null && _events.ReviewBonusPending ? ScoreBonus : 0);

    /// <summary>Live hard-metric faces (the host already owns the
    /// sim; the solver reads, never models).</summary>
    public int DeptsOf() => _intake != null ? _intake.OpenSlots : 0;
    public int ReputationOf() => _intake != null ? _intake.Reputation : 0;
    public int CureRatePctOf()
    {
        if (_intake == null || _intake.PatientTotal <= 0) return 0;
        return _intake.CuredTotal * 100 / _intake.PatientTotal;
    }
}

```

```

/// <summary>The reputation band (the frozen section-four
/// partition; the street flow rides TrafficAt, never re-model).</summary>
public ReputationBand BandFace()
    => _intake != null ? HospitalData.BandFor(_intake.Reputation)
       : HospitalData.BandFor(HospitalData.ReputationMin);

/// <summary>Static dual-track solver (the headless-gate face:
/// verdict math is pure, no runtime state).</summary>
public static ReviewVerdict Solve(ReviewGateDef gate, int depts, int cureRatePct, int reputation, int inspectScore)
{
    ReviewVerdict v;
    v.hardOk = gate != null
        && depts >= gate.minDepts
        && cureRatePct >= gate.minCureRatePct
        && reputation >= gate.minReputation;
    v.scoreOk = inspectScore >= 0; // a net non-negative inspection record
    v.pass = gate != null && v.hardOk && v.scoreOk; // dual-track law: both tracks together
    return v;
}

/// <summary>Requests a review against the current gate: refuses
/// while the cooldown window holds (fail = the next inspection
/// cycle) or at R5. On resolve books the settlement (rep swing
/// +-3), the ceremony / remediation flags and the evt_promote /
/// evt_inspect funnel lines. A pass promotes the ladder AND folds
/// the swap into both hosts inside the settle: the intake economy
/// row and the events stage pool read the new rank from the next
/// tick on (one verdict, three faces agree).</summary>
public ReviewVerdict RequestReview()
{
    var gate = CurrentGate;
    if (gate == null || !ReviewWindowOpen) { var none = new ReviewVerdict(); return none; }
    ReviewVerdict v = Solve(gate, DeptsOf(), CureRatePctOf(), ReputationOf(), InspectScore);
    _lastVerdictRank = _rank;
    _lastPass = v.pass;
    if (_intake != null)
        _intake.SetReputation(_intake.Reputation + (v.pass ? HospitalData.ReviewRepSwing : -HospitalData.ReviewRepSwing));
    if (v.pass)
    {
        _rank = gate.toRank;
        if (_intake != null) _intake.PromoteToRank(gate.toRank); // the host fold: the economy row swaps with the dean
        if (_events != null) _events.SetRank(gate.toRank); // the host fold: the stage pool widens with the rank
        _promotions++;
        _ceremonyArmed = true;
        _funnel.Add("evt_promote@r" + gate.toRank + ":" + gate.reviewEventId);
    }
    else
    {
        _failedReviews++;
        _remediationArmed = true;
    }
}

```

```

        _cooldownUntil = _t + ReviewCooldownSeconds; // re-review = the next inspection cycle
        _funnel.Add("evt_inspect@r" + gate.fromRank + ":" + gate.reviewEventId);
    }
    return v;
}

/// <summary>Advances the engine clock and tracks the reputation
/// band face: a band change books the visible comedy flags and
/// the evt_band_change funnel line (banner growth at the top
/// band, bad-review comedy at the bottom - the two-way face of
/// the second numeric line).</summary>
public void Advance(double dt)
{
    if (dt <= 0) return;
    _t += dt;
    var band = BandFace();
    if (band.id != _bandId)
    {
        _bandId = band.id;
        _badReviewComedyArmed = band.id == "rep_bad";
        _bannerGrowthArmed = band.id == "rep_banner";
        _funnel.Add("evt_band_change@" + band.id);
    }
}

/// <summary>Gate faces (headless checks drive the engine without
/// a live intake/events pair).</summary>
public void ArmForGate(int rank, int reputation)
{
    _rank = rank < 1 ? 1 : rank;
    if (_intake != null) _intake.SetReputation(reputation);
    _cooldownUntil = -1;
}

/// <summary>Canonical fingerprint: the zero-RNG parity
/// instrument (same script = same bytes).</summary>
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("t=").Append(_t.ToString("0.###"));
    sb.Append("|rank=").Append(_rank);
    sb.Append("|promo=").Append(_promotions);
    sb.Append("|fail=").Append(_failedReviews);
    sb.Append("|last=").Append(_lastVerdictRank).Append('/').Append(_lastPass ? 1 : 0);
    sb.Append("|score=").Append(InspectScore);
    sb.Append("|cd=").Append(_cooldownUntil.ToString("0.###"));
    sb.Append("|flags=").Append(_ceremonyArmed ? 1 : 0).Append(_remediationArmed ? 1 : 0)
        .Append(_badReviewComedyArmed ? 1 : 0).Append(_bannerGrowthArmed ? 1 : 0);
    sb.Append("|band=").Append(_bandId);
    sb.Append("|fun=").Append(_funnel.Count);
}

```

```

        return sb.ToString();
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\HospitalSave.cs =====
// HospitalSave.cs - the save system (G09-T014, G06SaveSystem / G12SaveSystem
// paradigm): single-key JSON persistence through the Sy.Core save service.
// Local storage is the only authority (red line 12, zero-server doctrine).
//
// Faces:
// - HospitalProfile: the save payload model. Only earned state is stored -
//   gold, rank, reputation, the department slot choice history (replayable),
//   the doctor card collection in hire order, the three-volume album rows,
//   research spend and the booked event chain flags. Duty roster state is
//   NEVER stored: it derives from the collection + open slot count on every
//   SyncRoster call, so a drifted duty row is structurally impossible
//   (derive-live zero-drift doctrine, G12 unlock-face sibling).
// - HospitalSaveSystem: four-state load (NewGame / Loaded / Migrated /
//   CorruptedReset). Version-migration skeleton: a payload without the
//   version key migrates by filling defaults for missing fields; a payload
//   from a newer build degrades to a fresh profile instead of guessing at
//   its schema. Corrupted rows drop with warnings, never fatal - a save can
//   brick a run only by refusing to load, never by throwing.
// - HospitalSaveSchema: editor-check verdict faces (pure C#, zero platform
//   coupling beyond ISaveService itself) so the Editor check rides these
//   faces instead of naming Sy.Core types directly (G12SaveSchema
//   paradigm). Faces mirror the save DoD: round-trip + four states +
//   migration defaults + tamper clamps + NG+ carry reconciliation against
//   HospitalEnding.SolveCarry / ApplyCarry.
using System;
using System.Collections.Generic;
using Sy.Core;

namespace Sy.CrazyHospital
{
    /// <summary>Outcome of a save-slot load (G06 four-state paradigm).</summary>
    public enum HospitalLoadStatus
    {
        NewGame,           // slot empty: fresh dean, nothing written yet
        Loaded,            // current-version payload applied as-is
        Migrated,          // older payload upgraded in place (defaults filled)
        CorruptedReset     // unreadable or future-version payload: profile reset
    }

    /// <summary>
    /// The G09 save payload (task T014): every earned face of the S2
    /// runtime in one serializable bag. Slot choices are the replayable
    /// history (pick WHAT opened, in open order) - the slot count, the
    /// last choice and the duty roster all derive from it live.
    /// </summary>

```

```

public sealed class HospitalProfile
{
    public int GoldHc; // the wallet (gold hundredths)
    public int Rank = 1; // R1..R5 ladder state
    public int Reputation; // 0-100 street fame
    public List<int> SlotChoices = new List<int>(); // department choice history
    public int PullCount; // doctor card draws taken
    public int DupeTotal; // duplicate cards folded
    public int ResearchSpendHc; // gold sunk into wonder drugs
    public List<string> DoctorIds = new List<string>(); // hire-order collection
    public List<string> AlbumIds = new List<string>(); // the three-volume unlocked rows
    public List<string> EventFlags = new List<string>(); // booked event chain flags

    /// <summary>Derived live from the replay history, never stored twice.</summary>
    public int OpenSlots => SlotChoices.Count;
}

/// <summary>
/// Single-key JSON persistence through the Sy.Core save service
/// (local authority, red line 12). All reads clamp to the frozen
/// schema laws (HospitalData) - a tampered payload can only ever
/// produce a schema-legal profile.
/// </summary>
public static class HospitalSaveSystem
{
    public const string Slot = "g09_main";
    public const string VersionKey = "g09v";
    public const int CurrentVersion = 1;

    /// <summary>Serialize the profile into the single-key JSON payload.</summary>
    public static string Serialize(HospitalProfile profile)
    {
        if (profile == null) throw new ArgumentNullException(nameof(profile));
        var choices = new List<object>();
        if (profile.SlotChoices != null)
            foreach (var c in profile.SlotChoices) choices.Add((long)c);
        var docs = new List<object>();
        if (profile.DoctorIds != null)
            foreach (var id in profile.DoctorIds) docs.Add(id);
        var albums = new List<object>();
        if (profile.AlbumIds != null)
            foreach (var id in profile.AlbumIds) albums.Add(id);
        var flags = new List<object>();
        if (profile.EventFlags != null)
            foreach (var id in profile.EventFlags) flags.Add(id);
        var data = new Dictionary<string, object>
        {
            { VersionKey, (long)CurrentVersion },
            { "gold", (long)profile.GoldHc },
            { "rank", (long)profile.Rank },
        }
    }
}

```

```

        { "rep", (long)profile.Reputation },
        { "choices", choices },
        { "pulls", (long)profile.PullCount },
        { "dupes", (long)profile.DupeTotal },
        { "rshc", (long)profile.ResearchSpendHc },
        { "docs", docs },
        { "albums", albums },
        { "flags", flags },
    };
    return MiniJson.Serialize(data);
}

/// <summary>Write the profile through the save service (local authority).</summary>
public static void Persist(ISaveService save, HospitalProfile profile, string slot = Slot)
{
    if (save == null) return;
    save.Save(slot, Serialize(profile));
}

/// <summary>
/// Load the slot into the given profile. Never throws on bad data:
/// corrupted or future-version payloads reset the profile to fresh
/// defaults. NewGame and CorruptedReset both reset deterministically
/// first, so a stale caller-side profile never survives either fresh
/// outcome.
/// </summary>
public static HospitalLoadStatus LoadInto(ISaveService save, HospitalProfile profile, string slot = Slot)
{
    if (save == null) throw new ArgumentNullException(nameof(save));
    if (profile == null) throw new ArgumentNullException(nameof(profile));

    string raw;
    try
    {
        raw = save.Load(slot);
    }
    catch (Exception e)
    {
        // the service itself chokes on a corrupted payload - degrade, never crash the boot
        Logger.Warn("G09.Save", "corrupt payload at service level, resetting slot: " + e.Message);
        return Fresh(profile);
    }
    if (string.IsNullOrEmpty(raw))
    {
        ResetProfile(profile);
        return HospitalLoadStatus.NewGame;
    }

    Dictionary<string, object> data;
    try

```

```

    {
        data = MiniJson.Deserialize(raw) as Dictionary<string, object>;
    }
    catch (Exception e)
    {
        Logger.Warn("G09.Save", "corrupt payload, resetting slot: " + e.Message);
        return Fresh(profile);
    }
    if (data == null)
    {
        Logger.Warn("G09.Save", "non-dict payload, resetting slot");
        return Fresh(profile);
    }

    int version = ReadInt(data, VersionKey, 0);
    if (version > CurrentVersion)
    {
        Logger.Warn("G09.Save", "future version " + version + " > " + CurrentVersion + ", resetting slot");
        return Fresh(profile);
    }

    profile.GoldHc = ClampMin(ReadInt(data, "gold", 0), 0); // wallet law: spend refuses before negative
    profile.Rank = Clamp(ReadInt(data, "rank", 1), 1, HospitalData.RankCount);
    profile.Reputation = Clamp(ReadInt(data, "rep", 0),
        HospitalData.ReputationMin, HospitalData.ReputationMax);
    profile.SlotChoices = ReadInts(data, "choices");
    profile.PullCount = ClampMin(ReadInt(data, "pulls", 0), 0);
    profile.DupeTotal = ClampMin(ReadInt(data, "dupes", 0), 0);
    profile.ResearchSpendHc = ClampMin(ReadInt(data, "rshc", 0), 0);
    profile.DoctorIds = ReadIds(data, "docs");
    profile.AlbumIds = ReadIds(data, "albums");
    profile.EventFlags = ReadIds(data, "flags");

    return version < CurrentVersion ? HospitalLoadStatus.Migrated : HospitalLoadStatus.Loaded;
}

static HospitalLoadStatus Fresh(HospitalProfile profile)
{
    ResetProfile(profile);
    return HospitalLoadStatus.CorrruptedReset;
}

static void ResetProfile(HospitalProfile profile)
{
    profile.GoldHc = 0;
    profile.Rank = 1;
    profile.Reputation = 0;
    profile.SlotChoices = new List<int>();
    profile.PullCount = 0;
    profile.DupeTotal = 0;
}

```

```

    profile.ResearchSpendHc = 0;
    profile.DoctorIds = new List<string>();
    profile.AlbumIds = new List<string>();
    profile.EventFlags = new List<string>();
}

static int Clamp(int v, int lo, int hi) => v < lo ? lo : (v > hi ? hi : v);
static int ClampMin(int v, int lo) => v < lo ? lo : v;

static int ReadInt(Dictionary<string, object> data, string key, int fallback)
{
    if (!data.TryGetValue(key, out var value) || value == null) return fallback;
    try { return Convert.ToInt32(Convert.ToDouble(value)); }
    catch { return fallback; }
}

static List<int> ReadInts(Dictionary<string, object> data, string key)
{
    var list = new List<int>();
    if (!data.TryGetValue(key, out var value) || !(value is System.Collections.IEnumerable rows)) return list;
    foreach (var row in rows)
    {
        int v;
        try { v = Convert.ToInt32(Convert.ToDouble(row)); }
        catch
        {
            Logger.Warn("G09.Save", "non-int choice row, dropped");
            continue;
        }
        if (v < 0)
        {
            Logger.Warn("G09.Save", "negative choice row, dropped");
            continue;
        }
        list.Add(v);
    }
    return list;
}

/// <summary>Id rows: null / empty / duplicate entries drop with a
/// warning (a save can never inflate a collection face).</summary>
static List<string> ReadIds(Dictionary<string, object> data, string key)
{
    var list = new List<string>();
    var seen = new HashSet<string>();
    if (!data.TryGetValue(key, out var value) || !(value is System.Collections.IEnumerable rows)) return list;
    foreach (var row in rows)
    {
        string id = row as string;
        if (string.IsNullOrEmpty(id))
    
```

```

        {
            Logger.Warn("G09.Save", key + " row without id, dropped");
            continue;
        }
        if (!seen.Add(id))
        {
            Logger.Warn("G09.Save", key + " duplicate row, dropped: " + id);
            continue;
        }
        list.Add(id);
    }
    return list;
}

}

/// <summary>
/// Editor-check verdict faces for the save domain (G12SaveSchema
/// paradigm): pure C#, EditMode-pinnable, mirroring the save DoD in
/// machine-readable form so the SaveCheck gate owns round-trip + four
/// states + migration defaults + tamper clamps + the NG+ carry
/// reconciliation against HospitalEnding.
/// </summary>
public static class HospitalSaveSchema
{
    /// <summary>In-memory ISaveService stub (check path only, zero platform coupling).</summary>
    sealed class MemorySaveService : ISaveService
    {
        readonly Dictionary<string, string> slots = new Dictionary<string, string>();

        public void Save(string slot, string payloadJson) { slots[slot] = payloadJson; }

        public string Load(string slot)
        {
            string v;
            return slots.TryGetValue(slot, out v) ? v : null;
        }

        public SyncResult CloudSync() { return new SyncResult(false, null); }
    }

    static HospitalProfile MakeSeededProfile()
    {
        return new HospitalProfile
        {
            GoldHc = 12345,
            Rank = 3,
            Reputation = 62,
            SlotChoices = new List<int> { 0, 2, 1 },
            PullCount = 7,
            DupeTotal = 2,
        }
    }
}

```

```

        ResearchSpendHc = 350,
        DoctorIds = new List<string> { "doc_threesec", "doc_wangdali" },
        AlbumIds = new List<string> { "dis_hiccup_bug", "drug_hiccup_cola" },
        EventFlags = new List<string> { HospitalEvents.ChainReviewBonus, HospitalEvents.ChainBanner },
    };
}

/// <summary>Serialize - persist - reload keeps every field and re-serializes identically.</summary>
public static bool RoundTripPreservesPayload()
{
    var save = new MemorySaveService();
    var seeded = MakeSeededProfile();
    string once = HospitalSaveSystem.Serialize(seeded);
    HospitalSaveSystem.Persist(save, seeded);
    var back = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, back) != HospitalLoadStatus.Loaded) return false;
    if (HospitalSaveSystem.Serialize(back) != once) return false;
    return back.GoldHc == 12345 && back.Rank == 3 && back.Reputation == 62
        && back.OpenSlots == 3 && back.PullCount == 7 && back.DupeTotal == 2
        && back.ResearchSpendHc == 350
        && back.DoctorIds.Count == 2 && back.DoctorIds[0] == "doc_threesec"
        && back.AlbumIds.Count == 2 && back.AlbumIds[0] == "dis_hiccup_bug"
        && back.EventFlags.Count == 2;
}

/// <summary>Empty slot = NewGame, and the reset wipes any stale caller-side rows.</summary>
public static bool StateNewGameOnEmptySlot()
{
    var save = new MemorySaveService();
    var stale = MakeSeededProfile();
    if (HospitalSaveSystem.LoadInto(save, stale) != HospitalLoadStatus.NewGame) return false;
    return stale.GoldHc == 0 && stale.Rank == 1 && stale.OpenSlots == 0
        && stale.DoctorIds.Count == 0 && stale.AlbumIds.Count == 0 && stale.EventFlags.Count == 0;
}

/// <summary>Version-less payload (pre-version build) migrates by filling defaults.</summary>
public static bool StateMigratedOnVersionlessPayload()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot,
        "{ \"gold\":900, \"rank\":2, \"rep\":40, \"choices\":[0], \"docs\":[\"doc_wangdali\"], \"albums\":[], \"flags\":[] }");
    var profile = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, profile) != HospitalLoadStatus.Migrated) return false;
    // the missing keys (pulls / dupes / rshc) migrate to their defaults
    return profile.GoldHc == 900 && profile.Rank == 2 && profile.Reputation == 40
        && profile.OpenSlots == 1 && profile.PullCount == 0 && profile.DupeTotal == 0
        && profile.ResearchSpendHc == 0
        && profile.DoctorIds.Count == 1 && profile.DoctorIds[0] == "doc_wangdali";
}

```

```

/// <summary>Unparseable payload degrades to a fresh profile instead of crashing the boot.</summary>
public static bool StateCorruptedResetOnGarbage()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot, "{ not json at all}");
    var stale = MakeSeededProfile();
    if (HospitalSaveSystem.LoadInto(save, stale) != HospitalLoadStatus.CorruptedReset) return false;
    return stale.GoldHc == 0 && stale.OpenSlots == 0 && stale.DoctorIds.Count == 0;
}

/// <summary>A payload from a newer build resets rather than guessing at its schema.</summary>
public static bool StateCorruptedResetOnFutureVersion()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot, "{\"g09v\":99,\"gold\":1}");
    var profile = new HospitalProfile();
    return HospitalSaveSystem.LoadInto(save, profile) == HospitalLoadStatus.CorruptedReset
        && profile.GoldHc == 0;
}

/// <summary>A tampered payload can only ever produce a schema-legal
/// profile: the frozen clamps hold and collection rows dedupe.</summary>
public static bool TamperClampsToSchemaLaws()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot,
        "{\"g09v\":1,\"gold\":-500,\"rank\":9,\"rep\":250,\"choices\":[ -1,2],\"pulls\":-4,\""
        + "\"docs\":[\"doc_wangdali\",\"doc_wangdali\",\"\"],\"albums\":[\"dis_monday\"],\"flags\":[ ]}\"");
    var profile = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, profile) != HospitalLoadStatus.Loaded) return false;
    return profile.GoldHc == 0
        && profile.Rank == HospitalData.RankCount
        && profile.Reputation == HospitalData.ReputationMax
        && profile.OpenSlots == 1 && profile.SlotChoices[0] == 2
        && profile.PullCount == 0
        && profile.DoctorIds.Count == 1 && profile.DoctorIds[0] == "doc_wangdali";
}

/// <summary>NG+ carry reconciliation (task DoD: the inherit face
/// matches HospitalEnding): the carry pack folded from the saved
/// profile's faces keeps gold 20% floored and the album rows, rank
/// and event flags reset by simply NOT being carried, and applying
/// the pack onto a fresh album replants the same ids with zero drift.</summary>
public static bool NgPlusCarryMatchesEndingFaces()
{
    var save = new MemorySaveService();
    var seeded = MakeSeededProfile();
    HospitalSaveSystem.Persist(save, seeded);
    var back = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, back) != HospitalLoadStatus.Loaded) return false;

```

```

    // fresh run faces: album replanted from the saved rows, doctor desk null
    // (the doctor id leg rides the profile round-trip, the desk's own
    // InheritFrom idempotence is DoctorCheck territory)
    var freshAlbum = new HospitalAlbum(null);
    var carry = HospitalEnding.SolveCarry(back.GoldHc, AlbumFromProfile(back), null);
    if (carry.GoldCarryHc != HospitalEnding.SolveGoldCarryHc(back.GoldHc)) return false;
    if (carry.GoldCarryHc != back.GoldHc * HospitalEnding.GoldCarryPercent / 100) return false;
    if (carry.DoctorIds == null || carry.DoctorIds.Count != 0) return false;

    var ending = new HospitalEnding();
    ending.ApplyCarry(carry, freshAlbum, null);
    // the replanted album rows match the saved rows exactly (zero drift)
    if (freshAlbum.Unlocked.Count != back.AlbumIds.Count) return false;
    for (int i = 0; i < back.AlbumIds.Count; i++)
        if (freshAlbum.Unlocked[i] != back.AlbumIds[i]) return false;
    // the ending books its one carry line, and the once-guard facade face stays untouched
    if (ending.FacadeLit) return false;

    // the fresh run's own save profile is a new-game identity: rank / flags never carry
    var freshProfile = new HospitalProfile();
    if (freshProfile.Rank != 1 || freshProfile.EventFlags.Count != 0) return false;
    return true;
}

/// <summary>Duty roster state derives live from the collection +
/// slot count (SyncRoster), never from the save payload - a drifted
/// duty row is structurally impossible.</summary>
public static bool DutyRosterDerivesZeroDrift()
{
    var desk = new HospitalDoctors(null, null, () => "gate_day");
    desk.InheritFrom(new List<string> { "doc_threesec", "doc_wangdali" });
    desk.SyncRoster(1);
    if (desk.Roster.Count != 1) return false;
    desk.SyncRoster(4);
    if (desk.Roster.Count != 2) return false;
    // the save payload carries no roster field at all: the serialized
    // seed contains no duty face, only the collection ids
    string json = HospitalSaveSystem.Serialize(MakeSeededProfile());
    return json.Contains("docs") && !json.Contains("duty") && !json.Contains("roster");
}

static HospitalAlbum AlbumFromProfile(HospitalProfile profile)
{
    var album = new HospitalAlbum(null);
    if (profile != null && profile.AlbumIds != null) album.InheritFrom(profile.AlbumIds);
    return album;
}
}
}

```

```

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\HospitalShareCard.cs =====
// HospitalShareCard.cs - the top-grade stamp certificate share card
// (G09-T011 prefix, SYS08 / R5 ending face): the brag card a stamped
// dean shows off - the grade title, four stat lines, the honor-wall
// photo row and the challenge footer - composed through the shared
// AutoCardComposer face (Sy.Social; template key g09_sharecard:
// missing art falls back to the procedural dark card, headless stays
// green). Every delivered share books evt_album_share; a compose that
// never lands books nothing (the funnel only ever counts real shares).
// The honor-wall law is closed-set: one stamp cell (tier = rank 1-5)
// plus one cell per album volume (tier = rate/20 floored: 0 empty,
// 100% gold). Pure C# + the shared composer, zero RNG, zero scene
// state, every copy routed through HospitalTexts keys.
using System.Collections.Generic;
using Sy.Social;
using UnityEngine;

namespace Sy.CrazyHospital
{
    public sealed class HospitalShareCard
    {
        public const string TemplateKey = "g09_sharecard";
        public const string EventId = "evt_album_share";
        public const string QueryPrefix = "g09";
        public const int WallCells = 4; // stamp + three album volumes
        public const int PinnedComposeBudgetBytes = 8 * 1024 * 1024; // canary budget pins the 1080x1440 tier

        readonly List<string> _funnel = new List<string>();
        public IReadOnlyList<string> Funnel => _funnel;

        /// <summary>Honor-wall tier solver (closed set): rate 0-19 -> 0
        /// (empty cell), each +20% one tier up, 100% = gold tier 5,
        /// out-of-range rates clamp.</summary>
        public static int SolveWallTier(int ratePct)
            => Mathf.Clamp(ratePct / 20, 0, 5);

        /// <summary>The collection-depth face of the share payload: the
        /// sum of wall tiers (the query carries it as vols).</summary>
        public static int VolumeTierSum(int[] volumeRates)
        {
            int n = 0;
            if (volumeRates != null) for (int i = 0; i < volumeRates.Length && i < 3; i++) n += SolveWallTier(volumeRates[i]);
            return n;
        }

        /// <summary>Pure spec solver (the headless face): title = the
        /// grade name of the rank, lines = rep / cure / depts / on-duty,
        /// footer = the challenge, wall = stamp + three volume cells in
        /// pack order (diseases / doctors / drugs).</summary>
        public static ShareCard SolveSpec(int rank, int reputation, int cureRatePct,

```

```

    int depts, int doctorsOnDuty, int[] volumeRates)
{
    rank = Mathf.Clamp(rank, 1, 5);
    var rates = volumeRates ?? new int[0];
    var cells = new CardCell[WallCells];
    cells[0] = new CardCell { Tier = rank, Stack = 1 };
    for (int i = 0; i < 3; i++)
        cells[i + 1] = new CardCell { Tier = rates.Length > i ? SolveWallTier(rates[i]) : 0, Stack = 1 };
    var lines = new[]
    {
        HospitalTexts.EnText(HospitalTexts.KeyStatReputation) + ": " + reputation,
        HospitalTexts.EnText(HospitalTexts.KeyStatCureRate) + ": " + Mathf.Max(0, cureRatePct) + "%",
        HospitalTexts.EnText(HospitalTexts.KeyStatDepts) + ": " + Mathf.Max(0, depts),
        HospitalTexts.EnText(HospitalTexts.KeyDutyRoster) + ": " + Mathf.Max(0, doctorsOnDuty),
    };
    var query = QueryPrefix + "&rank=" + rank + "&rep=" + reputation
        + "&cure=" + Mathf.Max(0, cureRatePct) + "&depts=" + Mathf.Max(0, depts)
        + "&docs=" + Mathf.Max(0, doctorsOnDuty) + "&vols=" + VolumeTierSum(rates);
    var card = AutoCardComposer.Card(TemplateKey,
        HospitalTexts.EnText("rank_" + rank), lines,
        HospitalTexts.EnText(HospitalTexts.KeyShareChallenge), query);
    card.Formation = cells;
    card.FormationCols = WallCells;
    card.FormationRows = 1;
    return card;
}

/// <summary>Live fold face: reads the wired faces and folds into
/// the pure solver (the host owns the sim, this face only reads;
/// null faces fold to the honest zeros face).</summary>
public static ShareCard BuildSpec(HospitalRank rank, HospitalAlbum album, HospitalDoctors doctors)
{
    var rates = new int[3];
    if (album != null)
        for (int i = 0; i < 3; i++) rates[i] = album.RatePctOf(HospitalAlbum.VolumeIds()[i]);
    return SolveSpec(
        rank != null ? rank.Rank : 1,
        rank != null ? rank.ReputationOf() : 0,
        rank != null ? rank.CureRatePctOf() : 0,
        rank != null ? rank.DeptsOf() : 0,
        doctors != null ? doctors.CollecteCount : 0,
        rates);
}

/// <summary>Shares the certificate: builds the spec, composes
/// the PNG through the shared RT face and books evt_album_share
/// only on a delivered card (Ok=false composes book nothing).
/// Returns the compose verdict for the caller to surface.</summary>
public ComposeResult Share(HospitalRank rank, HospitalAlbum album, HospitalDoctors doctors,
    int maxPngBytes = ShareCardService.MaxPngBytes)

```

```

    {
        var card = BuildSpec(rank, album, doctors);
        var res = new ShareCardService().Compose(card, maxPngBytes);
        if (res != null && res.Ok)
            _funnel.Add(EventId + "@r" + (rank != null ? rank.Rank : 1) + ":" + res.PngBytes);
        return res;
    }

    /// <summary>Canonical fingerprint: the zero-RNG parity
    /// instrument (same script = same bytes).</summary>
    public string Fingerprint()
    {
        var sb = new System.Text.StringBuilder();
        sb.Append("fun=").Append(_funnel.Count);
        for (int i = 0; i < _funnel.Count; i++) sb.Append('|').Append(_funnel[i]);
        return sb.ToString();
    }
}

// ===== FILE: Assets\Game\Scripts\Runtime\Game\HospitalStage.cs =====
// HospitalStage.cs - the opening beat driver (G09-T005).
// Drives HospitalIntake through the frozen red-line-11 opening script and
// stamps every promise on the sim clock: the facade hook owns t0, the first
// department decision lands inside the 3-8s window, the queue head registers
// half a second later and the first coin fires visual+sfx+number on one
// tick, then the first fictional patient portrait with the absurd diagnosis
// line (the first laugh) and the first review preview card land inside the
// 10s window. Pure C# zero-RNG: the same script always fingerprints
// byte-identical (Redline11Check consumes this face).
using System.Collections.Generic;

namespace Sy.CrazyHospital
{
    public enum HospitalPhase
    {
        Hook, // the facade + the street queue caption
        ClinicOpen, // the first department decision made, doors open
        FirstCoin, // the first registration coin landed
        FirstLaughter, // the absurd diagnosis portrait fired (the first laugh)
    }

    public sealed class HospitalStage
    {
        // the frozen opening script (red-line-11 beat anchors)
        public const double DecisionWindowStart = 3.0; // T+3s: the doors decision face goes live
        public const double FirstLaughAt = 8.5; // T+8.5s: patient portrait + absurd diagnosis
        public const double ReviewPreviewAt = 9.5; // T+9.5s: the first review preview card

        readonly HospitalIntake _intake;
        readonly HospitalEvents _events;
    }
}

```

```

readonly HospitalRank _rank; // T008: the rank-review gate rides the opening clock
readonly List<(string id, double t)> _stamps = new List<(string, double)>();
double _t;
bool _deptOpen;
double _deptOpenedAt = -1;
bool _firstCoin;
bool _admitBlocked; // the registration attempt refused this call: retry on the next Advance (per-call law)
bool _laughDone;
bool _previewDone;
string _firstPatientDiag = "";
int _firstCoinGoldBefore;
int _firstCoinJumpHc;

HospitalStage(HospitalIntake intake)
{
    _intake = intake;
    _events = new HospitalEvents(intake, 1); // T007: the event engine rides the opening clock (own timeline - stamps stay
untouched)
    _rank = new HospitalRank(intake, _events, 1); // T008: the rank gate rides the same clock (identity: no write until a
review resolves)
}

/// <summary>The fresh dean: an unlicensed clinic, one bed, three
/// patients already queuing at the street corner.</summary>
public static HospitalStage NewDean()
    => new HospitalStage(new HospitalIntake(1));

// ---- live faces ----
public double SimTime => _t;
public HospitalPhase Phase
{
    get
    {
        if (_laughDone) return HospitalPhase.FirstLaughter;
        if (_firstCoin) return HospitalPhase.FirstCoin;
        if (_deptOpen) return HospitalPhase.ClinicOpen;
        return HospitalPhase.Hook;
    }
}
public HospitalIntake Intake => _intake;
public HospitalEvents Events => _events;
public HospitalRank Rank => _rank;
public bool ModalActive => false; // zero login wall, zero blocking modal (red line 11)
public bool CanDecide => _deptOpen == false; // the doors decision is live right after the hook
public IReadOnlyList<(string id, double t)> Stamps => _stamps;

/// <summary>First stamp time of an event id (-1 when never fired).</summary>
public double TimeOf(string id)
{
    for (int i = 0; i < _stamps.Count; i++)
        if (_stamps[i].id == id) return _stamps[i].t;
    return -1;
}

```

```

    }

    void Stamp(string id)
    {
        _stamps.Add((id, _t));
    }

    /// <summary>Advances the opening clock: drives the intake economy
    /// and fires the scheduled beats (registration admit, first laugh,
    /// review preview) on the frozen script. The dt is sliced at the
    /// next scheduled beat so every promise lands on its scripted sim
    /// time no matter how coarse the caller drives the clock (a whole
    /// 13s hop still books the +0.5s admit on time - the bed turn then
    /// completes before a review at t+13, the ladder walk rides this).</summary>
    public void Advance(double dt)
    {
        if (dt <= 0) return;
        if (TimeOf("evt_hospital_hook") < 0)
            Stamp("evt_hospital_hook"); // the hook owns t0 (facade + street queue + caption)

        _admitBlocked = false; // the registration retry law is per-call (a refused admit waits for the next Advance)
        double remaining = dt;
        while (remaining > 0)
        {
            double seg = remaining;
            double next = NextBeatTime();
            if (next - _t < seg) seg = next - _t; // slice at the beat, never past it
            if (seg > 0)
            {
                _t += seg;
                _intake.Advance(seg);
                _events.Advance(seg);
                _rank.Advance(seg); // T008: the reputation-band face tracks the live rep (own funnel, never writes the
economy)
                remaining -= seg;
            }
            FireScheduledBeats();
            if (seg <= 0) remaining = 0; // a due beat just resolved: hand control back to the caller, never spin inside one
call
        }
    }

    /// <summary>The earliest pending scheduled beat (double.MaxValue
    /// when the script has nothing left to fire). The blocked-admit
    /// face keeps a refused registration out of the slice plan until
    /// the next Advance call retries it (the old per-call law).</summary>
    double NextBeatTime()
    {
        double next = double.MaxValue;
        if (_deptOpen && !_firstCoin && !_admitBlocked)
        {
            double admit = _deptOpenedAt + HospitalIntake.FirstAdmitDelaySeconds;

```

```

        if (admit < next) next = admit;
    }
    if (_deptOpen && !_laughDone && FirstLaughAt < next) next = FirstLaughAt;
    if (_deptOpen && !_previewDone && ReviewPreviewAt < next) next = ReviewPreviewAt;
    return next;
}

/// <summary>Fires every scheduled beat whose scripted time the
/// clock has reached.</summary>
void FireScheduledBeats()
{
    // the queue head steps in half a second after the doors open
    if (_deptOpen && !_firstCoin && !_admitBlocked
        && _t >= _deptOpenedAt + HospitalIntake.FirstAdmitDelaySeconds)
    {
        _firstCoinGoldBefore = _intake.GoldHc;
        if (_intake.TryRegisterQueueHead())
        {
            _firstCoinJumpHc = _intake.GoldHc - _firstCoinGoldBefore; // snapshot at the stamp tick
            Stamp("evt_first_registration");
            Stamp("coin_visual"); // the coin popup face
            Stamp("coin_sfx"); // the signature coin chime trigger
            Stamp("coin_number"); // the gold counter jump
            Stamp("evt_first_coin");
            _firstCoin = true;
            _firstPatientDiag = HospitalTexts.EnText(HospitalTexts.KeyDiagHiccupBug);
        }
        else _admitBlocked = true; // queue or bed refused: retry on the next Advance call
    }

    // the first laugh: the first fictional patient portrait + the absurd diagnosis line
    if (_deptOpen && !_laughDone && _t >= FirstLaughAt)
    {
        Stamp("evt_first_laugh");
        Stamp("evt_first_patient_portrait");
        Stamp("evt_first_diag_line");
        _laughDone = true;
    }

    // the first review preview card (the level-1 review teaser)
    if (_deptOpen && !_previewDone && _t >= ReviewPreviewAt)
    {
        Stamp("evt_review_preview");
        _previewDone = true;
    }
}

/// <summary>The doors decision: opens the first preset department
/// slot (pick WHAT opens). Returns false while the wallet or the
/// rank chain refuses.</summary>

```

```

public bool OpenFirstDepartment(int choice)
{
    if (_deptOpen) return false;
    if (TimeOf("evt_hospital_hook") < 0) return false;    // the hook owns to
    if (!_intake.OpenNextSlot(choice)) return false;
    _deptOpen = true;
    _deptOpenedAt = _t;
    Stamp("evt_dept_opened");
    Stamp("evt_first_decision");
    return true;
}

public int FirstCoinJumpHc => _firstCoin ? _firstCoinJumpHc : 0;
public string FirstPatientDiag => _firstPatientDiag;

/// <summary>Requests the rank review (T008 host face): the engine
/// owns the verdict and both host folds (the economy row swap and
/// the stage pool widen land inside the settle). The opening script
/// never calls this - the first review is a player action, so the
/// signature law holds.</summary>
public ReviewVerdict RequestReview() => _rank.RequestReview();

/// <summary>Opening fingerprint: every stamp, the wallet and the
/// intake state - byte-identical across identical scripts.</summary>
public string OpeningSig()
{
    var sb = new System.Text.StringBuilder();
    foreach (var s in _stamps) sb.Append(s.id).Append('@').Append(s.t.ToString("0.###")).Append(';');
    sb.Append(_intake.Fingerprint());
    return sb.ToString();
}
}
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Game\HospitalTexts.cs =====
// HospitalTexts.cs - the hospital copy law for Crazy Hospital (G09-T004).
// English source strings live here as compiled code (the family source law:
// code strings stay English); the zh-CN localization rides a separate data
// file (Assets/_Game/Configs/Texts_zhCN.json - data, not source). Lookup
// follows the shared LocaleService contract: a missing key falls back to the
// key itself (machine-readable, never a crash), and a zh entry missing from
// the data file falls back to the English source. Zero scattered copy: every
// UI-facing sentence routes through these keys - DataSchemaCheck scans the
// runtime tree for stray CJK and proves the coverage law. The medical
// disclaimer is a permanent resident key (compliance face, always on).
using System;
using System.Collections.Generic;
using Sy.Core;

namespace Sy.CrazyHospital
{

```

```

public static class HospitalTexts
{
    // ---- key constants (the compiled copy surface) ----
    public const string KeyHookCaption = "hook_caption";
    public const string KeyMedicalDisclaimer = "medical_disclaimer"; // the permanent compliance face
    public const string KeyRank1 = "rank_1";
    public const string KeyRank2 = "rank_2";
    public const string KeyRank3 = "rank_3";
    public const string KeyRank4 = "rank_4";
    public const string KeyRank5 = "rank_5";
    public const string KeyStatPatients = "stat_patients";
    public const string KeyStatReputation = "stat_reputation";
    public const string KeyStatCureRate = "stat_cure_rate";
    public const string KeyStatGold = "stat_gold";
    public const string KeyStatDepts = "stat_depts";
    public const string KeyDeptOpen = "dept_open";
    public const string KeyDeptSlots = "dept_slots";
    public const string KeyRegFee = "reg_fee";
    public const string KeyBedFee = "bed_fee";
    public const string KeyRepBad = "rep_bad";
    public const string KeyRepPlain = "rep_plain";
    public const string KeyRepGood = "rep_good";
    public const string KeyRepBanner = "rep_banner";
    public const string KeyReviewLevel1 = "review_level1";
    public const string KeyReviewLevel2 = "review_level2";
    public const string KeyReviewLevel3 = "review_level3";
    public const string KeyReviewTop = "review_top";
    public const string KeyGatePromote = "gate_promote";
    public const string KeyGateRectify = "gate_rectify";
    public const string KeyResearchTitle = "research_title";
    public const string KeyDrugHiccupCola = "drug_hiccup_cola";
    public const string KeyDrugSleepwalkPatch = "drug_sleepwalk_patch";
    public const string KeyDrugShockSugar = "drug_shock_sugar";
    public const string KeyDisHiccupBug = "disease_hiccup_bug";
    public const string KeyDisGummyBone = "disease_gummy_bone";
    public const string KeyDisMonday = "disease_monday";
    public const string KeyDisHeadGrass = "disease_head_grass";
    public const string KeyDisCactusAccident = "disease_cactus_accident";
    public const string KeyDiagHiccupBug = "diag_hiccup_bug";
    public const string KeyDiagGummyBone = "diag_gummy_bone";
    public const string KeyDiagMonday = "diag_monday";
    public const string KeyDiagHeadGrass = "diag_head_grass";
    public const string KeyDiagCactusAccident = "diag_cactus_accident";
    public const string KeyAlbumTitle = "album_title";
    public const string KeyOfflineSummary = "offline_summary";
    public const string KeyOfflineDouble = "offline_double";
    public const string KeyNightTide = "night_tide";
    public const string KeyCarryLabel = "carry_label";
    public const string KeyNewJourney = "new_journey";
    public const string KeyNewJourneyConfirm = "new_journey_confirm";
}

```

```

public const string KeyShareChallenge = "share_challenge";
public const string KeyDailyReport = "daily_report";
// ---- the quack-doctor hiring face (T006, SYS03) ----
public const string KeyDoctorDraw = "doctor_draw";
public const string KeyDoctorOnStaff = "doctor_on_staff";
public const string KeyDutyRoster = "duty_roster";
public const string KeyDocThreesec = "doc_threesec";
public const string KeyDocRemedyQuack = "doc_remedy_quack";
public const string KeyDocMoyu = "doc_moyu";
public const string KeyDocWangdali = "doc_wangdali";
public const string KeyDocNightNurse = "doc_night_nurse";
public const string KeyDocBeautyChief = "doc_beauty_chief";
public const string KeyQuirkThreesec = "quirk_threesec";
public const string KeyQuirkRemedyQuack = "quirk_remedy_quack";
public const string KeyQuirkMoyu = "quirk_moyu";
public const string KeyQuirkWangdali = "quirk_wangdali";
public const string KeyQuirkNightNurse = "quirk_night_nurse";
public const string KeyQuirkBeautyChief = "quirk_beauty_chief";
// ---- the event-engine face (T007, SYS04): card titles, the
// inspection preview and the two choice labels ----
public const string KeyEventPreview = "event_preview";
public const string KeyEventChoiceGo = "event_choice_go";
public const string KeyEventChoiceSlack = "event_choice_slack";
public const string KeyEventE01 = "event_e01";
public const string KeyEventE02 = "event_e02";
public const string KeyEventE03 = "event_e03";
public const string KeyEventE04 = "event_e04";
public const string KeyEventE05 = "event_e05";
public const string KeyEventE06 = "event_e06";
public const string KeyEventE07 = "event_e07";
public const string KeyEventE08 = "event_e08";
public const string KeyEventE09 = "event_e09";
public const string KeyEventE10 = "event_e10";
// ---- the rank-review face (T008, SYS05/SYS06): the promotion
// ceremony show and the rectification comedy show beats ----
public const string KeyCeremonyShow = "ceremony_show";
public const string KeyRectifyShow = "rectify_show";

/// <summary>English source table (compiled - the source-of-law face).</summary>
public static readonly Dictionary<string, string> En = new Dictionary<string, string>
{
    { KeyHookCaption, "WELCOME TO CRAZY HOSPITAL - YOUR DEAN LIFE STARTS NOW" },
    { KeyMedicalDisclaimer, "FOR ENTERTAINMENT ONLY - NOT MEDICAL ADVICE" },
    { KeyRank1, "UNLICENSED CLINIC" },
    { KeyRank2, "LEVEL-1 HOSPITAL" },
    { KeyRank3, "LEVEL-2 HOSPITAL" },
    { KeyRank4, "LEVEL-3 HOSPITAL" },
    { KeyRank5, "TOP-GRADE STAMPED" },
    { KeyStatPatients, "PATIENTS" },
    { KeyStatReputation, "REPUTATION" },

```

```

{ KeyStatCureRate, "CURE RATE" },
{ KeyStatGold, "GOLD" },
{ KeyStatDepts, "DEPTS" },
{ KeyDeptOpen, "OPEN A DEPARTMENT" },
{ KeyDeptSlots, "SLOTS: {0}/{1}" },
{ KeyRegFee, "REG FEE" },
{ KeyBedFee, "BED FEE" },
{ KeyRepBad, "REVIEW STORM - THE STREETS GO QUIET" },
{ KeyRepPlain, "QUIET STREETS, BUSINESS AS USUAL" },
{ KeyRepGood, "STEADY PATIENTS, STEADY LINES" },
{ KeyRepBanner, "BANNER-WALL QUEUES OUT THE DOOR" },
{ KeyReviewLevel1, "LEVEL-1 REVIEW (THE PAPERWORK LITERATURE PRIZE)" },
{ KeyReviewLevel2, "LEVEL-2 REVIEW (THE MYSTERY PATIENT STRIKES)" },
{ KeyReviewLevel3, "LEVEL-3 REVIEW (THE GREAT HYGIENE INSPECTION)" },
{ KeyReviewTop, "TOP-GRADE REVIEW (THE REHEARSED MIRACLE SHOW)" },
{ KeyGatePromote, "PROMOTED! {0} UNLOCKED" },
{ KeyGateRectify, "RECTIFY AND RESUBMIT - NEXT REVIEW WINDOW SOON" },
{ KeyResearchTitle, "WONDER-DRUG LAB" },
{ KeyDrugHiccupCola, "HICCUP COLA - THE BURP GETS LOUDER" },
{ KeyDrugSleepwalkPatch, "SLEEPWALK PATCH - WALK MORE, DREAM DEEPER" },
{ KeyDrugShockSugar, "SHOCK SUGAR - FULL-BODY PIXEL JITTER" },
{ KeyDisHiccupBug, "THE HICCUP BUG" },
{ KeyDisGummyBone, "GUMMY-BONE SYNDROME" },
{ KeyDisMonday, "MONDAY SYNDROME" },
{ KeyDisHeadGrass, "GRASS ON THE HEAD (CODER EDITION)" },
{ KeyDisCactusAccident, "CACTUS-DIET ACCIDENT" },
{ KeyDiagHiccupBug, "DIAGNOSIS: PRESCRIBE ONE MORE HICCUP COLA." },
{ KeyDiagGummyBone, "DIAGNOSIS: BONE IS NOW DESSERT. AVOID DESSERT." },
{ KeyDiagMonday, "DIAGNOSIS: SEVERE MONDAY. COME BACK ON FRIDAY." },
{ KeyDiagHeadGrass, "DIAGNOSIS: WATER TWICE A DAY, NO MOWING." },
{ KeyDiagCactusAccident, "DIAGNOSIS: THE CACTUS IS FINE. THE PATIENT IS NOT." },
{ KeyAlbumTitle, "FICTIONAL DISEASE ALBUM" },
{ KeyOfflineSummary, "WHILE YOU WERE GONE..." },
{ KeyOfflineDouble, "DOUBLE IT?" },
{ KeyNightTide, "NIGHT EMERGENCY TIDE!" },
{ KeyCarryLabel, "NEXT-DEAN CARRY: {0}" },
{ KeyNewJourney, "START A NEW DEAN LIFE" },
{ KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT CLINIC" },
{ KeyShareChallenge, "CAN YOUR HOSPITAL BEAT MINE?" },
{ KeyDailyReport, "DAY {0}: {1}" },
// ---- the quack-doctor hiring face (T006): names, mischief beats
// and the spree lines - pure comedy, zero medical claims ----
{ KeyDoctorDraw, "HIRE 10 DOCTORS (AD-FUNDED)" },
{ KeyDoctorOnStaff, "ALREADY ON PAYROLL - THE JOKE HIRES TWICE" },
{ KeyDutyRoster, "ON DUTY" },
{ KeyDocThreesec, "THREE-SECOND DOCTOR" },
{ KeyDocRemedyQuack, "FOLK-REMEDY QUACK" },
{ KeyDocMoyu, "SLACKING DIRECTOR" },
{ KeyDocWangdali, "WANG-DALI THE BONE BOLTER" },
{ KeyDocNightNurse, "NIGHT-SHIFT NURSE" },

```

```

{ KeyDocBeautyChief, "BEAUTY-INSTITUTE CHIEF" },
{ KeyQuirkThreesec, "NEXT! - THE THREE-SECOND CONSULT, QUEUE MOVES ITSELF" },
{ KeyQuirkRemedyQuack, "ANCESTRAL REMEDY, MYSTICAL EFFECTS - NO GUARANTEES, OBVIOUSLY" },
{ KeyQuirkMoyu, "THE VANISHING WARD ROUND - NOBODY SAW HIM ALL DAY" },
{ KeyQuirkWangdali, "BONES BOLTED ON LIKE SCREWS - DO NOT ASK" },
{ KeyQuirkNightNurse, "SLEEPWALKS THE FLAG PATROL AT 3 A.M. - NEVER TIRED, EVER" },
{ KeyQuirkBeautyChief, "WANTED A STAR FACE, GOT A PIXEL FACE" },
// ---- the event-engine face (T007): the comedy lives in the
// titles - five inspection cards + five daily cards ----
{ KeyEventPreview, "THE INSPECTORS ARE COMING - EVERYONE LOOK BUSY!" },
{ KeyEventChoiceGo, "PLAY IT STRAIGHT" },
{ KeyEventChoiceSlack, "WING IT" },
{ KeyEventE01, "SURPRISE HYGIENE INSPECTION (EVERYONE PRETEND TO WORK)" },
{ KeyEventE02, "THE LEADERSHIP VISIT (THE REHEARSED MIRACLE SHOW)" },
{ KeyEventE03, "THE MYSTERY PATIENT (A SERVICE-ATTITUDE EXAM)" },
{ KeyEventE04, "REVIEW-PAPER LITERATURE PRIZE (NEGATIVE-GROWTH OPTIMIZATION)" },
{ KeyEventE05, "FIRE-DRILL STRETCHER RELAY (WANG-DALI CARRIES THREE)" },
{ KeyEventE06, "THE SLEEPWALKING HERO ROUNDS THE WARD AND HANGS A BANNER" },
{ KeyEventE07, "THE HICCUP KID STARTS A HOSPITAL-WIDE HICCUP TIDE" },
{ KeyEventE08, "THE CACTUS REGULAR (A SPINY ATTRIBUTION COMEDY)" },
{ KeyEventE09, "THE OVERTIMED CODER GROWS GRASS" },
{ KeyEventE10, "THE MIRROR PRINCESS ORDERS A STAR FACE (GETS A PIXEL FACE)" },
// ---- the rank-review face (T008): the two show beats ----
{ KeyCeremonyShow, "THE STAMP CEREMONY! FIREWORKS OVER THE COURTYARD, A RAIN OF BANNERS" },
{ KeyRectifyShow, "QUACK-REMEDY SIDE EFFECT - THE REVIEW EXPERT CHECKS INTO OUR OWN WARD 3" },
};

/// <summary>English lookup (missing key falls back to the key itself
/// - machine-readable, never a crash).</summary>
public static string EnText(string key)
    => En.TryGetValue(key, out var v) && !string.IsNullOrEmpty(v) ? v : key;

/// <summary>Parses the zh-CN data file payload into a key-to-text
/// map (values must be non-empty strings; anything else drops the
/// entry so the lookup degrades to English, never to garbage).</summary>
public static Dictionary<string, string> ParseZh(string json)
{
    var map = new Dictionary<string, string>();
    if (string.IsNullOrEmpty(json)) return map;
    try
    {
        if (!MiniJson.Deserialize(json) is Dictionary<string, object> d) return map;
        foreach (var kv in d)
            if (!string.IsNullOrEmpty(kv.Key) && kv.Value is string s && !string.IsNullOrEmpty(s))
                map[kv.Key] = s;
    }
    catch
    {
        // corrupt data file = empty map = English fallback face
        return new Dictionary<string, string>();
    }
}

```

```
    }
    return map;
}

/// <summary>LocaleService-compatible lookup delegate over a parsed
/// zh map: zh-CN hits read the data file, everything else (and any
/// miss) reads the English source. Wire-ready for the T005 UI face.</summary>
public static Func<string, string, string> LookupWithZh(Dictionary<string, string> zh)
{
    return (lang, key) =>
    {
        if (lang == "zh-CN" && zh != null && zh.TryGetValue(key, out var v) && !string.IsNullOrEmpty(v))
            return v;
        return EnText(key);
    };
}
}
```